

Análisis multivariado — ATUS ZMM

Estructura de covarianza, correlación, factorización y estacionalidad de los accidentes de tránsito de la Zona Metropolitana de Monterrey, 2019-2024.

Fuente: `data/processed/atus_zmm_limpio.parquet` (379,294 registros × 71 columnas). Ese archivo **no está versionado** — se genera con:

```
uv run limpiar-atus
```

Las columnas en MAYÚSCULAS son del INEGI; las minúsculas las construye `geostats.limpieza`. Ver [docs/limpieza.md](#).

Qué hay aquí

§	Tema
1	Qué variables admiten este tratamiento y cuáles no
2	Sesgo y curtosis
3	Normalidad univariada
4	Normalidad multivariada — Mardia y Henze-Zirkler
5	Matriz de varianzas-covarianzas
6	Correlación: Pearson, Spearman y Kendall
7	Cualitativas: one-hot y V de Cramér
8	Análisis factorial: KMO, Bartlett, PCA y EFA con varimax
9	Serie de tiempo: descomposición STL

Advertencia que atraviesa todo el documento. Con $n = 379,294$ cualquier prueba de hipótesis rechaza: una desviación irrelevante de la normalidad produce un p de cero. Los contrastes se reportan por completitud, pero **lo que se lee es el tamaño del efecto** — sesgo, curtosis, magnitud de las correlaciones—, no el p .

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from scipy import stats
from sklearn.decomposition import PCA, FactorAnalysis
```

```

from statsmodels.tsa.seasonal import STL

from geostats import limpieza, rutas

d = pd.read_parquet(rutas.ATUS_ZMM_LIMPIO)
N = len(d)
print(f"{N:,} registros x {len(d.columns)} columnas | "
      f"{d.fecha.min():%Y-%m-%d} a {d.fecha.max():%Y-%m-%d} | "
      f"{d.CVE_MUN.nunique()} municipios")

```

379,294 registros x 71 columnas | 2019-01-01 a 2024-12-31 | 18 municipios

Estilo de las gráficas

Misma identidad que `calidad_datos.qmd`. Dos añadidos que este notebook necesita:

- **Escala divergente** para las correlaciones. Una correlación tiene polaridad ($-1 \dots 0 \dots +1$), así que pide dos tonos con un **gris neutro** en el centro: Azul Prusia para lo negativo, Rojo profundo para lo positivo. Nunca un tono en el punto medio y nunca arcoíris. El par azul/rojo se validó: ΔE 18.9 bajo protanopia y 23.9 en visión plena, sobre un piso de 15.
- **Escala secuencial** de un solo tono para magnitudes sin signo (V de Cramér, varianza explicada), donde el orden es lo único que hay que leer.

```

# --- Identidad GeoStats -----
AZUL      = "#005991" # Azul Prusia ajustado -> datos
ROJO      = "#8B2C1A" # Rojo profundo -> énfasis, títulos
OXIDO     = "#B15E2E" # Rojo óxido -> detalle cálido
GRAFITO   = "#2C2C2C" # texto y estructura
GRIS      = "#F2F2F2" # fondos, rejilla y punto medio de la divergente
BLANCO    = "#FFFFFF"
AZUL_RAMPA = ["#005991", "#1b77b8", "#4195d9"]

# Correlación = polaridad -> divergente, con gris neutro al centro.
DIVERGENTE = LinearSegmentedColormap.from_list("geostats_div", [AZUL, GRIS, ROJO])
# Magnitud sin signo -> un solo tono, claro a oscuro.
SECUENCIAL = LinearSegmentedColormap.from_list("geostats_seq", [BLANCO, AZUL])

TITULAR = ["Montserrat", "Helvetica Neue", "Arial", "DejaVu Sans"]
TEXTO   = ["Cormorant Garamond", "EB Garamond", "Georgia", "DejaVu Serif"]
CIFRAS  = ["Roboto Mono", "Menlo", "DejaVu Sans Mono"]

plt.rcParams.update({
    "figure.dpi": 130, "figure.facecolor": BLANCO, "axes.facecolor": BLANCO,
    "savefig.facecolor": BLANCO, "font.family": "sans-serif",
    "font.sans-serif": TITULAR, "font.serif": TEXTO, "font.monospace": CIFRAS,
    "font.size": 9, "axes.edgecolor": GRIS, "axes.labelcolor": GRAFITO,
    "axes.labelsize": 9, "axes.titlecolor": ROJO, "axes.titlesize": 11.5,
    "axes.titleweight": "bold", "axes.titlelocation": "left", "axes.titlepad": 16,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRIS,

```

```

"grid.linewidth": 1.0, "xtick.color": GRAFITO, "ytick.color": GRAFITO,
"xtick.labelsize": 8.5, "ytick.labelsize": 8.5, "legend.frameon": False,
"legend.fontsize": 8.5, "lines.linewidth": 1.8,
})

# Se llama `pulir` y no `limpiar` para no chocar con `geostats.limpieza.limpiar`.
def pulir(ax, ejes=("top", "right", "left")):
    for lado in ejes:
        ax.spines[lado].set_visible(False)
    ax.tick_params(length=0)
    for etiqueta in ax.get_xticklabels() + ax.get_yticklabels():
        etiqueta.set_fontfamily("monospace")
    return ax

```

1. Qué variables admiten este tratamiento

La base tiene 71 columnas, pero **casi ninguna es cuantitativa en el sentido que estas técnicas suponen**. La mayoría son códigos de catálogo (cualitativas disfrazadas de número) o conteos con exceso de ceros.

Un conteo que es cero en el 99.9 % de los registros no tiene estructura lineal que correlacionar: su covarianza con cualquier cosa está dominada por los ceros. Meterlo a una matriz de correlación o a un PCA produce números que **parecen resultados y no lo son**.

Así que el primer paso es medir, no elegir a mano. La regla:

Entra al análisis multivariado la variable cuantitativa con **menos de 99 % de ceros y desviación estándar > 0.05**.

```

CANDIDATAS = (
    ["edad", "hora", "n_vehiculos", "TOTMUERTOS", "TOTHERIDOS"]
    + limpieza.MUERTOS + limpieza.HERIDOS + limpieza.VEHICULOS
    + ["LONGITUD", "LATITUD"]
)

desc = []
for c in CANDIDATAS:
    v = pd.to_numeric(d[c], errors="coerce").astype("float64").dropna()
    desc.append({
        "variable": c, "no_nulos": len(v), "%ceros": (v == 0).mean() * 100,
        "media": v.mean(), "sd": v.std(),
        "sesgo": stats.skew(v), "curtosis": stats.kurtosis(v), # curtosis en exceso
    })
desc = pd.DataFrame(desc).set_index("variable")

# La regla, aplicada. `sd` se compara con 0.05 y los ceros con 99 %.
entra = (desc["%ceros"] < 99) & (desc["sd"] > 0.05)
CUANT = desc.index[entra].tolist()

```

```
print(f"{len(CANDIDATAS)} candidatas -> {len(CUANT)} entran, {(~entra).sum()} quedan fuera\n")
print("FUERA:")
print(desc.loc[~entra, ["%ceros", "sd", "sesgo", "curtosis"]].round(3).to_string())
```

30 candidatas -> 17 entran, 13 quedan fuera

FUERA:

	%ceros	sd	sesgo	curtosis
variable				
TOTMUERTOS	99.717	0.063	38.352	4086.438
CONDMUERTO	99.871	0.037	29.235	933.195
PASAMUERTO	99.960	0.031	157.575	46519.135
PEATMUERTO	99.884	0.035	30.647	995.159
CICLMUERTO	99.993	0.008	120.769	14583.231
OTROMUERTO	99.998	0.004	232.770	54179.857
CICLHERIDO	99.890	0.034	31.849	1115.870
OTROHERIDO	99.980	0.016	83.106	7766.383
MICROBUS	99.846	0.041	27.177	800.474
OMNIBUS	99.589	0.067	16.528	293.929
TRANVIA	100.000	0.000	NaN	NaN
FERROCARRI	99.930	0.026	37.793	1426.299
BICICLETA	99.555	0.067	15.373	250.725

```
print("DENTRO:")
desc.loc[CUANT, ["no_nulos", "%ceros", "media", "sd", "sesgo", "curtosis"]].round(3)
```

DENTRO:

	no_nulos	%ceros	media	sd	sesgo	curtosis
variable						
edad	273635	0.000	38.348	13.603	0.678	-0.030
hora	379294	2.421	13.129	5.563	-0.382	-0.470
n_vehiculos	379294	0.000	1.888	0.452	0.305	6.900
TOTHERIDOS	379294	94.556	0.068	0.342	11.908	342.027
CONDHERIDO	379294	97.596	0.026	0.172	7.590	73.908
PASAHHERIDO	379294	98.837	0.017	0.215	30.657	1671.907
PEATHERIDO	379294	97.756	0.023	0.157	7.259	61.540
AUTOMOVIL	379294	16.932	1.261	0.759	-0.121	-0.355
CAMPASAJ	379294	77.235	0.265	0.521	1.905	3.142
PASCAMION	379294	93.141	0.073	0.275	3.901	15.873
CAMIONETA	379294	91.343	0.093	0.312	3.486	13.168
CAMION	379294	95.649	0.044	0.211	4.731	22.268
TRACTOR	379294	95.842	0.043	0.208	4.917	24.585

	no_nulos	%ceros	media	sd	sesgo	curtosis
variable						
MOTOCICLET	379294	92.364	0.077	0.271	3.323	9.865
OTROVEHIC	379294	98.012	0.020	0.143	7.221	54.943
LONGITUD	379294	0.000	-100.306	0.090	-0.023	1.696
LATITUD	379294	0.000	25.708	0.068	0.082	5.174

TRANVIA es **constante cero** en la ZMM: desviación estándar exactamente 0. No es que aporte poco, es que su correlación con cualquier variable es una división entre cero. Sola, esa columna bastaría para volver singular la matriz.

TOTMUERTOS queda fuera por poco (99.72 % de ceros) y la exclusión incomoda, porque la mortalidad es justo lo que interesa en seguridad vial. La decisión es deliberada: con 1,075 accidentes fatales de 379,294, un coeficiente de Pearson contra esa columna mide casi solo la ausencia de muertos. **La severidad se analiza en la §7 como variable cualitativa (gravedad)**, que es donde sí se puede decir algo defendible.

2. Sesgo y curtosis

Para una normal, sesgo = 0 y curtosis en exceso = 0. Los criterios usuales piden $|\text{sesgo}| < 2$ y $|\text{curtosis}| < 7$ para tratar una variable como aproximadamente simétrica.

```
# Dos paneles, una métrica cada uno y el mismo orden de variables: cada nombre
# se lee en el eje y ninguna etiqueta se encima con otra.
#
# Escala symlog, no log: `edad`, `hora` y AUTOMOVIL tienen curtosis NEGATIVA.
# Recortarla a un positivo pequeño para poder usar log las dibujaría como si
# fueran casi cero por arriba, que es falso. symlog es lineal cerca del origen
# y logarítmica fuera, así que muestra el signo y el rango completo.
orden = desc["curtosis"].sort_values().index
color = [AZUL if v in CUANT else "#C9C9C9" for v in orden]
y = np.arange(len(orden))

fig, (a1, a2) = plt.subplots(1, 2, figsize=(9.6, 6.6), sharey=True)

a1.scatter(desc.loc[orden, "curtosis"], y, s=40, c=color, zorder=3)
a1.axvline(7, color=GRAFITO, lw=1, ls=(0, (4, 3)), zorder=2)
a1.set_xscale("symlog", linthresh=1)
a1.set_xlabel("curtosis en exceso (symlog)")
a1.set_title("Curtosis · umbral 7", fontsize=9.5, color=GRAFITO, pad=8)

a2.scatter(desc.loc[orden, "sesgo"], y, s=40, c=color, zorder=3)
for limite in (-2, 2):
    a2.axvline(limite, color=GRAFITO, lw=1, ls=(0, (4, 3)), zorder=2)
a2.set_xscale("symlog", linthresh=1)
a2.set_xlabel("sesgo (symlog)")
a2.set_title("Sesgo · umbral ±2", fontsize=9.5, color=GRAFITO, pad=8)
```

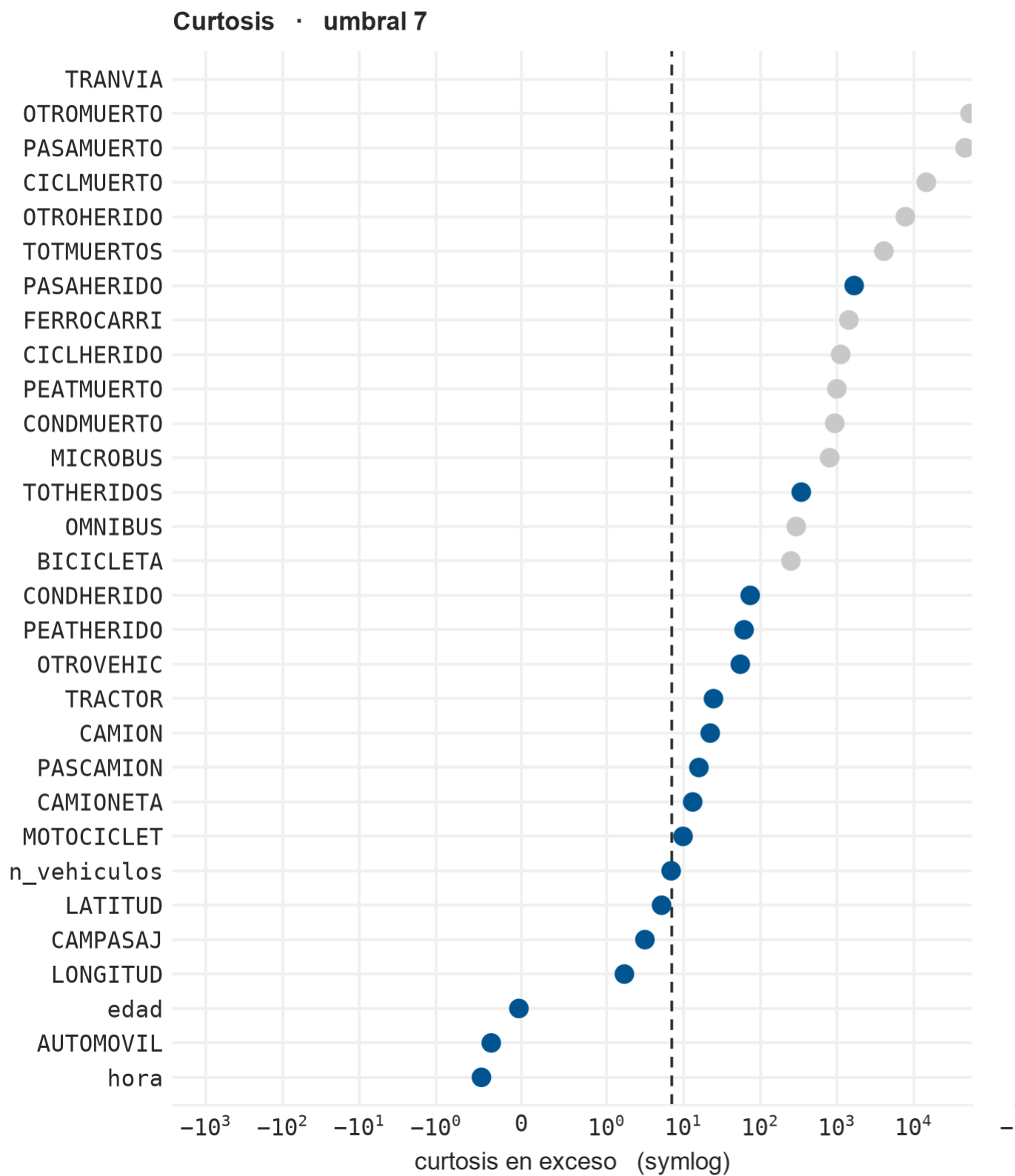
```

a1.set_yticks(y, orden)
for ax in (a1, a2):
    pulir(ax)
    ax.set_ylim(-.8, len(orden) - .2)

fig.suptitle("Casi ninguna variable es tratable como continua", x=0.005,
             ha="left", color=ROJO, fontweight="bold", fontsize=11.5)
fig.tight_layout()
fig.text(0.005, -0.02,
        f"En azul las {len(CUANT)} que pasan la regla; en gris las {(~entra).sum()} "
        f"descartadas. TRANVIA no aparece: al ser constante, su sesgo y su "
        f"curtosis no están definidos.",
        fontsize=8, color=GRAFITO, family="serif")
plt.show()

```

Casi ninguna variable es tratable como continua



En azul las 17 que pasan la regla; en gris las 13 descartadas. TRANVIA no aparece: al ser constante, su sesgo



El gráfico es el argumento de la §1 hecho imagen. OTROMUERTO tiene curtosis 54,180 y sesgo 233: es una columna con 62 unos a nivel nacional y ceros en todo lo demás. La curtosis no está midiendo la forma de

una distribución, está midiendo cuán rara es la excepción.

Siete de las 30 candidatas caen dentro de la región aceptable en **los dos** ejes: `edad`, `hora`, `AUTOMOVIL`, `n_vehiculos`, `CAMPASAJ`, `LONGITUD` y `LATITUD`. Pero pasar el umbral no las vuelve equivalentes, y conviene separarlas:

- `edad` — sesgo 0.68, curtosis -0.03. La única que es una medición continua de verdad, y la que más se parece a una normal. Tiene sentido: es la edad de una persona, no un conteo de sucesos.
- `hora` — sesgo -0.38, curtosis -0.47. Simétrica, pero **cíclica**: la hora 23 es adyacente a la 0 y ningún coeficiente lineal captura eso.
- `LONGITUD` y `LATITUD` — simétricas porque la mancha urbana lo es, no porque midan algo que se distribuya normal. Entran al análisis como control espacial, no como variables de interés.
- `AUTOMOVIL`, `n_vehiculos` y `CAMPASAJ` — conteos acotados y pequeños (0 a 9. que resultan casi simétricos. Pasan el umbral por tener poco rango, no por ser continuos: `n_vehiculos` se queda en curtosis 6.9, a un pelo del corte.

3. Normalidad univariada

Tres contrastes con supuestos distintos. Shapiro-Wilk está limitado a 5,000 observaciones, así que va sobre una muestra; D'Agostino y Anderson-Darling corren sobre todo el conjunto.

```
import warnings
# scipy 1.17 avisa que en 1.19 habrá que elegir `method` para el p-valor. Aquí
# se leen los valores críticos tabulados, que es la forma clásica de Anderson-
# Darling, así que el aviso no aplica y se silencia para no repetirlo 17 veces.
warnings.filterwarnings("ignore", category=FutureWarning)

rng = np.random.default_rng(20260915)
MUESTRA_SW = 5000

filas = []
for c in CUANT:
    v = pd.to_numeric(d[c], errors="coerce").astype("float64").dropna().to_numpy()
    sw = stats.shapiro(rng.choice(v, MUESTRA_SW, replace=False))
    k2, p_k2 = stats.normaltest(v)
    ad = stats.anderson(v, dist="norm")
    filas.append({
        "variable": c,
        "sesgo": stats.skew(v), "curtosis": stats.kurtosis(v),
        "Shapiro W": sw.statistic, "Shapiro p": sw.pvalue,
        "D'Agostino K2": k2, "K2 p": p_k2,
        "Anderson A2": ad.statistic, "critico 5%": ad.critical_values[2],
    })
norm = pd.DataFrame(filas).set_index("variable")
norm["rechaza AD"] = norm["Anderson A2"] > norm["critico 5%"]

print(f"Variables que NO rechazan normalidad (Anderson-Darling, 5%): "
```

```
f"{{~norm['rechaza AD']}.sum()}} de {{len(norm)}}\n"
norm.round(4)
```

Variables que NO rechazan normalidad (Anderson-Darling, 5%): 0 de 17

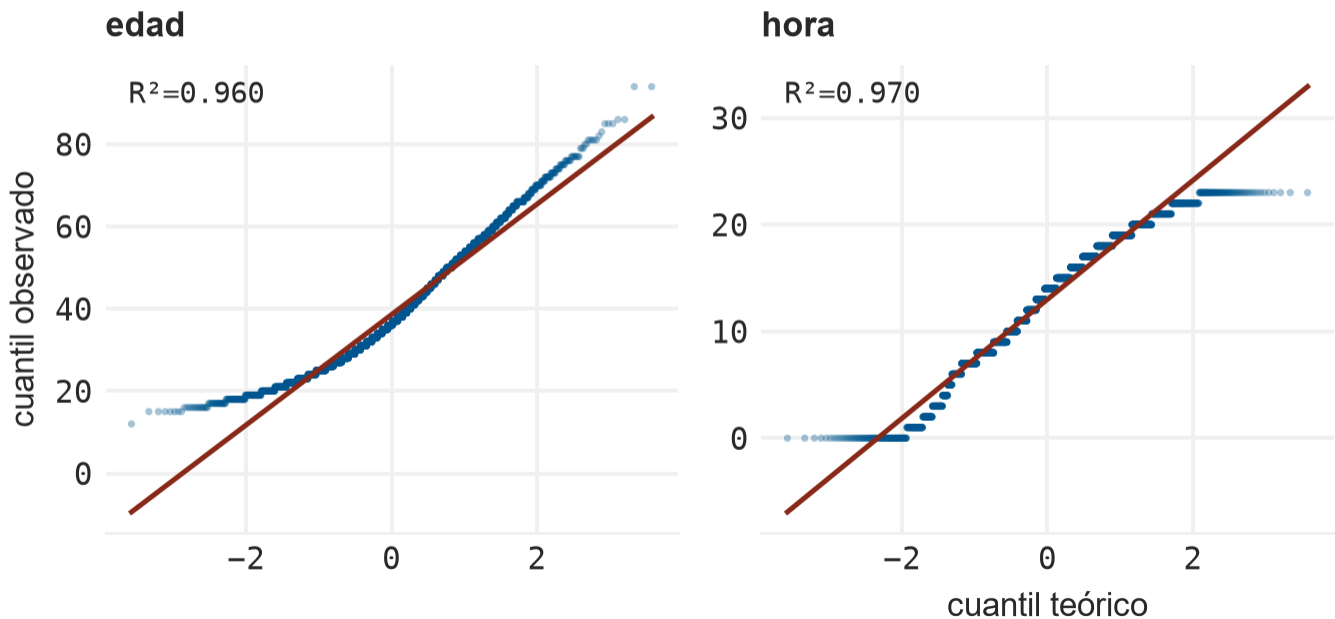
	sesgo	curtosis	Shapiro W	Shapiro p	D'Agostino K2	K2 p	Anderson A2	critico 5%	rechaza AD
variable									
edad	0.6781	-0.0303	0.9563	0.0	17402.3566	0.0	2966.7488	0.752	True
hora	-0.3816	-0.4696	0.9715	0.0	14736.7736	0.0	2406.1722	0.752	True
n_vehiculos	0.3052	6.9000	0.5636	0.0	57976.3084	0.0	83457.1199	0.752	True
TOTHERIDOS	11.9080	342.0267	0.2115	0.0	659531.6328	0.0	127574.4757	0.752	True
CONDERIDO	7.5901	73.9079	0.1238	0.0	498973.5812	0.0	141268.2548	0.752	True
PASAHERIDO	30.6565	1671.9075	0.0837	0.0	994365.0570	0.0	142137.9759	0.752	True
PEATHERIDO	7.2589	61.5402	0.1148	0.0	482169.5454	0.0	142169.6631	0.752	True
AUTOMOVIL	-0.1214	-0.3546	0.8261	0.0	3905.1809	0.0	29307.0968	0.752	True
CAMPASAJ	1.9050	3.1422	0.5456	0.0	133565.8892	0.0	82914.2865	0.752	True
PASCAMION	3.9012	15.8729	0.2864	0.0	300016.3198	0.0	130095.5457	0.752	True
CAMIONETA	3.4857	13.1677	0.3293	0.0	272406.2928	0.0	124489.3833	0.752	True
CAMION	4.7312	22.2679	0.1919	0.0	350580.6575	0.0	137758.3966	0.752	True
TRACTOR	4.9170	24.5846	0.1915	0.0	362219.7610	0.0	138048.9242	0.752	True
MOTOCICLET	3.3233	9.8646	0.2900	0.0	253200.4735	0.0	130043.7791	0.752	True
OTROVEHIC	7.2214	54.9432	0.1248	0.0	477137.5768	0.0	143062.2421	0.752	True
LONGITUD	-0.0227	1.6962	0.9692	0.0	14521.1438	0.0	3195.1323	0.752	True
LATITUD	0.0819	5.1744	0.9245	0.0	43035.5988	0.0	4908.1306	0.752	True

```
# Q-Q de las cuatro mas representativas: la mejor, una ciclica, un conteo y una
# coordenada. Un panel por variable: una sola serie cada uno, sin paleta
# categorica que inventar.
elegidas = ["edad", "hora", "n_vehiculos", "TOTHERIDOS"]
fig, ejes = plt.subplots(1, 4, figsize=(10.4, 2.9))

for ax, c in zip(ejes, elegidas):
    v = pd.to_numeric(d[c], errors="coerce").astype("float64").dropna()
    v = rng.choice(v.to_numpy(), 4000, replace=False)
    (osm, osr), (pend, orden, r) = stats.probplot(v, dist="norm")
    ax.scatter(osm, osr, s=4, color=AZUL, alpha=.35, zorder=3, edgecolors="none")
    ax.plot(osm, pend * osm + orden, color=ROJO, lw=1.4, zorder=4)
    ax.set_title(c, fontsize=9.5, color=GRAFITO, pad=8)
    ax.text(.04, .92, f"R²={r**2:.3f}", transform=ax.transAxes,
            family="monospace", fontsize=8, color=GRAFITO)
    ax.set_xlabel("cuantil teórico" if c == "hora" else "")
```

```
pulir(ax)
ejes[0].set_ylabel("cuantil observado")
fig.suptitle("Q-Q: ninguna es normal, pero los conteos ni se acercan",
             x=0.005, ha="left", color=ROJO, fontweight="bold", fontsize=11.5)
fig.tight_layout()
plt.show()
```

Q-Q: ninguna es normal, pero los conteos ni se acercan



Ninguna variable pasa Anderson-Darling, lo cual era previsible y **no es informativo**: con este n el contraste rechaza desviaciones que no cambiarían ninguna conclusión.

Lo que sí informa son los Q-Q. **edad** traza una S: sigue la recta en el cuerpo de la distribución y se separa en las dos colas ($R^2 = 0.960$). **n_vehiculos** (0.562) y **TOTHERIDOS** (0.210) son escaleras, conteos con dos o tres valores posibles en la práctica, imposibles de confundir con una continua.

hora alcanza el R^2 más alto de las cuatro (0.970) y eso **engaña**: la variable está discretizada en 24 valores y acotada entre 0 y 23, así que la nube forma escalones y se aplana contra los extremos. Que puntúe por encima de **edad** recuerda que el R^2 de un Q-Q mide ajuste a una recta, no normalidad. La curtosis casi nula de **edad** (-0.03) sigue siendo la mejor evidencia de cuál es la variable más tratable.

4. Normalidad multivariada — Mardia y Henze-Zirkler

Que cada variable no sea normal ya garantiza que el conjunto no lo es, pero la normalidad multivariada es un supuesto propio y se contrasta aparte. Es el supuesto que sostiene el análisis factorial de máxima verosimilitud y las pruebas sobre la matriz de covarianzas.

Ninguna de las dos pruebas viene en scipy, así que se programan. **Los dos requieren la matriz de distancias de Mahalanobis entre todos los pares**, que es $n \times n$: a 379,294 filas serían 1,072 GB. Van sobre muestras, repetidas varias veces para ver si el resultado es estable.

```
def mardia(X):
    """Sesgo y curtosis multivariados de Mardia."""
    X = np.asarray(X, dtype=float)
    n, p = X.shape
    c = X - X.mean(0)
    S = np.cov(c, rowvar=False, bias=True)          # estimador ML, como pide Mardia
    D = c @ np.linalg.pinv(S) @ c.T                 # Mahalanobis por pares
    b1p = (D ** 3).sum() / n ** 2
    b2p = (np.diag(D) ** 2).mean()
    g1 = p * (p + 1) * (p + 2) / 6
    chi2 = n * b1p / 6
    z = (b2p - p * (p + 2)) / np.sqrt(8 * p * (p + 2) / n)
    return {"b1p": b1p, "sesgo_p": stats.chi2.sf(chi2, g1),
            "b2p": b2p, "b2p_esperada": p * (p + 2),
            "curtosis_z": z, "curtosis_p": 2 * stats.norm.sf(abs(z))}

def henze_zirkler(X):
    """Prueba de Henze-Zirkler. Bajo H0 el estadístico es ~lognormal."""
    X = np.asarray(X, dtype=float)
    n, p = X.shape
    c = X - X.mean(0)
    Sinv = np.linalg.pinv(np.cov(c, rowvar=False, bias=True))
    Dij = np.einsum("ik,kl,jl->ij", c, Sinv, c)
    dii = np.diag(Dij)
    Dpar = dii[:, None] + dii[None, :] - 2 * Dij      # (xi-xj)' S^-1 (xi-xj)

    b = (1 / np.sqrt(2)) * ((2 * p + 1) * n / 4) ** (1 / (p + 4))
    a = 1 + 2 * b ** 2
    # El término central lleva (1+b^2) y solo el tercero (1+2b^2). Confundirlos
    # hace que la prueba rechace datos que sí son normales.
    uno_b = 1 + b ** 2
    hz = ((1 / n) * np.exp(-(b ** 2) / 2 * Dpar).sum()
          - 2 * uno_b ** (-p / 2) * np.exp(-(b ** 2) / (2 * uno_b) * dii).sum()
          + n * a ** (-p / 2))

    wb = (1 + b ** 2) * (1 + 3 * b ** 2)
    mu = 1 - a ** (-p / 2) * (1 + p * b ** 2 / a + p * (p + 2) * b ** 4 / (2 * a ** 2))
    si2 = (2 * (1 + 4 * b ** 2) ** (-p / 2)
           + 2 * a ** (-p) * (1 + 2 * p * b ** 4 / a ** 2
                               + 3 * p * (p + 2) * b ** 8 / (4 * a ** 4))
           - 4 * wb ** (-p / 2) * (1 + 3 * p * b ** 4 / (2 * wb)
                                   + p * (p + 2) * b ** 8 / (2 * wb ** 2)))

    pmu = np.log(np.sqrt(mu ** 4 / (si2 + mu ** 2)))
    psi = np.sqrt(np.log((si2 + mu ** 2) / mu ** 2))
    return {"HZ": hz, "p": stats.lognorm.sf(hz, psi, scale=np.exp(pmu))}
```

```
# Control: datos que SI son normales multivariados. Si una prueba los rechaza,
# está mal programada y cualquier resultado sobre los datos reales sobra.
ctrl = np.random.default_rng(7)
A = ctrl.normal(size=(6, 6))
SIM = ctrl.multivariate_normal(np.zeros(6), A @ A.T + np.eye(6), size=3000)

m, h = mardia(SIM), henze_zirkler(SIM)
print("CONTROL - normal multivariada simulada (no debe rechazar):")
print(f"  Mardia sesgo      b1p = {m['b1p']:8.4f}    p = {m['sesgo_p']:.3f}")
print(f"  Mardia curtosis    b2p = {m['b2p']:8.3f}    esperada {m['b2p_esperada']}")
print(f"    p = {m['curtosis_p']:.3f}")
print(f"  Henze-Zirkler      HZ = {h['HZ']:8.4f}    p = {h['p']:.3f}")
```

CONTROL - normal multivariada simulada (no debe rechazar):

```
Mardia sesgo      b1p =    0.1371    p = 0.121
Mardia curtosis    b2p =   48.182    esperada 48    p = 0.612
Henze-Zirkler      HZ =    0.9630    p = 0.734
```

```
# Los datos reales, sobre muestras repetidas.
X = d[CUANT].apply(pd.to_numeric, errors="coerce").astype("float64").dropna()
p_dim = X.shape[1]
print(f"matriz completa: {X.shape[0]:,} x {p_dim}\n")

REPS, N_MARDIA, N_HZ = 8, 4000, 2000
res = []
for r in range(REPS):
    idx = rng.choice(len(X), N_MARDIA, replace=False)
    S = X.to_numpy()[idx]
    m = mardia(S)
    h = henze_zirkler(S[:N_HZ])
    res.append({"muestra": r + 1, "b1p": m["b1p"], "sesgo p": m["sesgo_p"],
               "b2p": m["b2p"], "curtosis z": m["curtosis_z"],
               "HZ": h["HZ"], "HZ p": h["p"]})
res = pd.DataFrame(res).set_index("muestra")
print(f"b2p esperada bajo normalidad multivariada: p(p+2) = {p_dim*(p_dim+2)}")
res.round(3)
```

matriz completa: 273,635 x 17

b2p esperada bajo normalidad multivariada: $p(p+2) = 323$

	b1p	sesgo p	b2p	curtosis z	HZ	HZ p
muestra						
1	2604.262	0.0	3047.621	3389.923	45.707	0.0
2	4740.095	0.0	5143.333	5997.370	41.411	0.0
3	2484.778	0.0	3273.237	3670.631	42.854	0.0

	b1p	sesgo p	b2p	curtosis z	HZ	HZ p
muestra						
4	1242.773	0.0	1820.587	1863.270	43.149	0.0
5	1246.926	0.0	1851.791	1902.093	43.794	0.0
6	1158.999	0.0	1627.373	1622.877	42.467	0.0
7	1648.442	0.0	2190.932	2324.046	44.950	0.0
8	1232.348	0.0	1609.691	1600.877	42.516	0.0

El control confirma que las dos pruebas están bien programadas: sobre datos simulados genuinamente normales no rechazan (p de 0.12, 0.61 y 0.73).

Sobre los datos reales las ocho muestras rechazan sin excepción, y no es un rechazo marginal:

- **Sesgo de Mardia:** b_{1p} va de 1,159 a 4,740 según la muestra, cuando bajo normalidad debería ser ≈ 0 .
- **Curtosis de Mardia:** b_{2p} va de 1,610 a 5,143 contra un valor esperado de $p(p+2) = 323$: entre 5 y 16 veces el valor teórico.
- **Henze-Zirkler:** entre 41 y 46, unas cuarenta veces el 0.96 del control.

El estadístico oscila bastante entre muestras, y eso también informa: con colas tan pesadas, el valor depende de cuántos casos extremos cayeron en la muestra. La conclusión, en cambio, no se mueve.

Conclusión operativa: cualquier técnica que exija normalidad multivariada —factorial por máxima verosimilitud, contrastes sobre la matriz de covarianzas— queda descartada de entrada. Lo que sigue usa métodos que no la requieren: componentes principales (descriptivo, no inferencial), correlaciones de rango, y factorial por eje principal.

5. Matriz de varianzas-covarianzas

La covarianza **depende de las unidades**. Es su defecto y conviene verlo antes de pasar a la correlación, que es la misma matriz adimensional.

Dos comprobaciones antes de usarla:

1. **Borrado por lista o por pares.** De las 17 variables solo **edad** tiene nulos, así que ambas opciones dan casi lo mismo — pero el borrado por lista elimina 105,659 registros y ese descarte **no es aleatorio**: falta la edad en el 43.8 % de los accidentes fatales contra el 27.9 % de los de solo daños.
2. **Que la matriz sea semidefinida positiva.** Con borrado por pares cada celda se calcula sobre un subconjunto distinto y el resultado puede no serlo, lo que rompería el PCA y el factorial.

```
Xp = d[CUANT].apply(pd.to_numeric, errors="coerce").astype("float64")

print("faltante de `edad` según gravedad (por qué el borrado por lista sesga):")
print(d.assign(falta=d.edad.isna())
      .groupby("gravedad", observed=True)["falta"].mean().mul(100).round(1).to_string())
```

```

print()
for metodo, M in [("por pares", Xp.cov()), ("por lista", Xp.dropna().cov())]:
    w = np.linalg.eigvalsh(M.to_numpy())
    print(f"  {metodo:<10} n_efectivo={len(Xp) if metodo=='por pares' else len(Xp.dropna()):>8,}"
          f"  eigenvalor mínimo = {w.min():+.3e}"
          f"  {'PSD' if w.min() >= -1e-10 else 'NO PSD'}")

COV = Xp.cov()

```

faltante de `edad` según gravedad (por qué el borrado por lista sesga):

gravedad

Fatal 43.8

No fatal 26.5

Solo daños 27.9

```

por pares  n_efectivo= 379,294  eigenvalor mínimo = +3.166e-04  PSD
por lista  n_efectivo= 273,635  eigenvalor mínimo = +3.211e-04  PSD

```

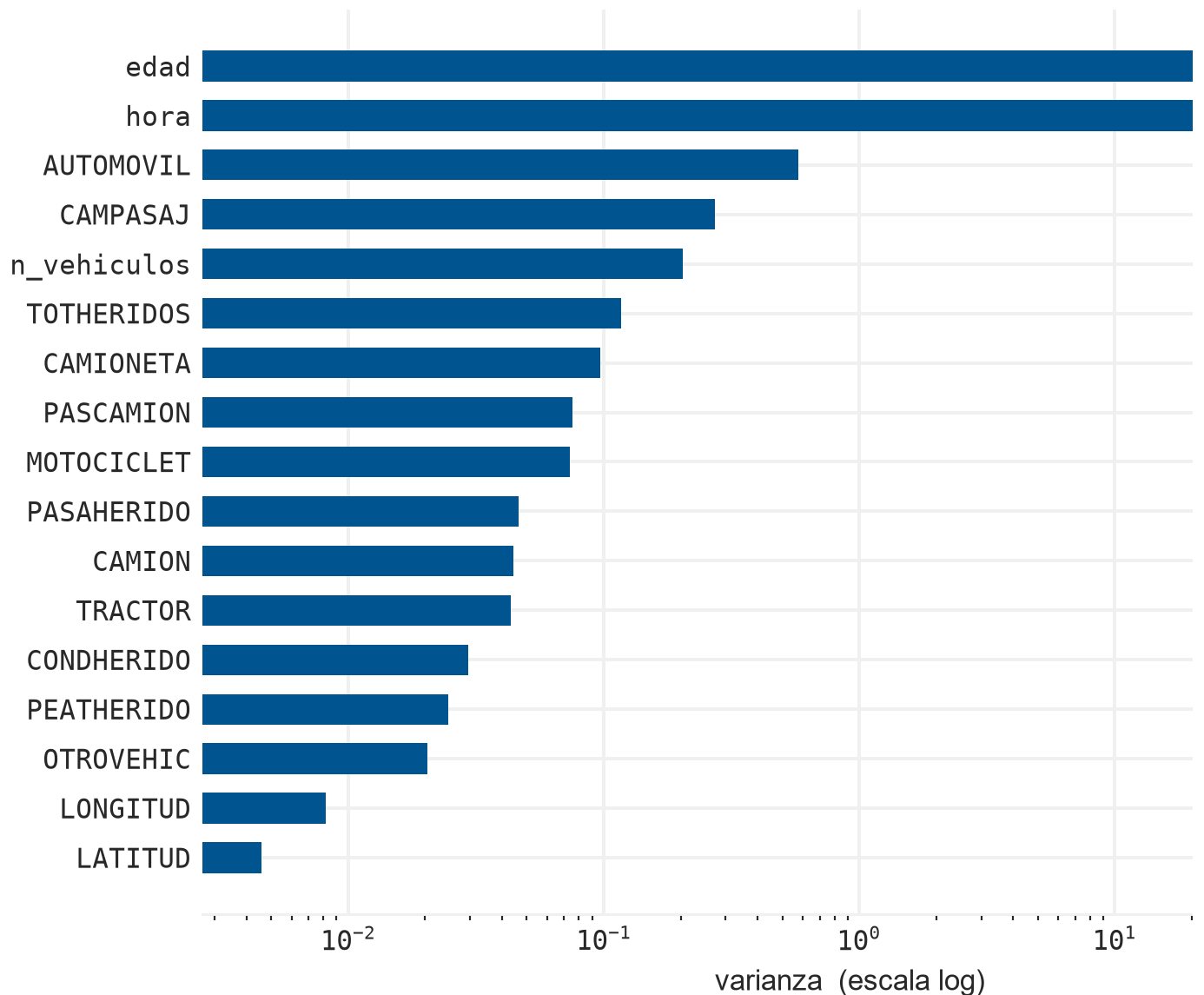
```

# La diagonal de la covarianza, que es la varianza de cada variable. En escala
# log porque abarca cinco órdenes de magnitud: eso es exactamente el problema.
var = np.diag(COV)
orden = np.argsort(var)
etiquetas = [CUANT[i] for i in orden]

fig, ax = plt.subplots(figsize=(7.4, 5.2))
y = np.arange(len(orden))
ax.barh(y, var[orden], height=.62, color=AZUL, zorder=3)
ax.set_xscale("log")
ax.set_yticks(y, etiquetas)
ax.set_xlabel("varianza (escala log)")
ax.set_title("La covarianza no es comparable entre variables")
pudir(ax, ejes=("top", "right", "left"))
fig.text(0.005, -0.02,
        f"De {var.min():.1e} en LATITUD a {var.max():.1f} en edad: un factor de "
        f"{var.max()/var.min():.0f}. Por eso se estandariza antes de correlacionar.",
        fontsize=8, color=GRAFITO, family="serif")
plt.show()

```

La covarianza no es comparable entre variables



De 4.6×10^{-3} en LATITUD a 185.0 en edad: un factor de 40,520. Por eso se estandariza antes de correlacionar



Las dos comprobaciones pasan: la matriz es semidefinida positiva con ambos métodos, así que el PCA y el factorial son viables. Se usa **borrado por pares**, que conserva las 379,294 filas donde la variable existe en lugar de descartar el 28 % de la base — y con ello, desproporcionadamente, los accidentes fatales.

La varianza va de 4.6×10^{-3} en LATITUD a 185 en edad: un factor de casi 40,000. Una matriz de covarianzas con estos rangos está dominada por edad y no dice nada sobre la estructura conjunta. De ahí en adelante, todo va estandarizado.

6. Correlación: Pearson, Spearman y Kendall

Tres coeficientes que miden cosas distintas:

	Mide	Supone
Pearson r	Asociación lineal	Escala de intervalo; sensible a valores extremos
Spearman ρ	Asociación monótona	Solo orden; robusto a extremos
Kendall τ	Concordancia de pares	Solo orden; mejor con muchos empates

Con conteos de exceso de ceros la distinción no es académica: hay empates por todas partes, que es justo el caso donde Kendall se comporta mejor que Spearman. Kendall es $O(n \log n)$ por par y con 379,294 filas $\times$ 136 pares no termina en un tiempo razonable, así que va sobre una muestra con semilla fija.

```
N_KENDALL = 8000

PEARSON = Xp.corr(method="pearson")
SPEARMAN = Xp.corr(method="spearman")
KENDALL = Xp.sample(N_KENDALL, random_state=20260915).corr(method="kendall")

print(f"Pearson y Spearman sobre {len(Xp):,} filas; Kendall sobre {N_KENDALL:,}.")

def pares(M):
    """Pares únicos ordenados por |coeficiente|, sin la diagonal."""
    m = M.where(np.triu(np.ones(M.shape), 1).astype(bool)).stack()
    return m.reindex(m.abs().sort_values(ascending=False).index)

print("\nDiez asociaciones lineales más fuertes (Pearson):")
print(pares(PEARSON).head(10).round(3).to_string())
```

Pearson y Spearman sobre 379,294 filas; Kendall sobre 8,000.

Diez asociaciones lineales más fuertes (Pearson):

TOTHERIDOS	PASAHERIDO	0.735
	CONDHERIDO	0.635
AUTOMOVIL	CAMPASAJ	-0.478
TOTHERIDOS	PEATHERIDO	0.449
n_vehiculos	AUTOMOVIL	0.447
CONDHERIDO	MOTOCICLET	0.265
AUTOMOVIL	CAMIONETA	-0.259
n_vehiculos	PEATHERIDO	-0.235
AUTOMOVIL	PASCAMION	-0.232
CONDHERIDO	PASAHERIDO	0.218

```
fig, ax = plt.subplots(figsize=(7.8, 6.6))

M = PEARSON.to_numpy().copy()
```

```

mascara = np.triu(np.ones_like(M, dtype=bool))    # solo el triángulo inferior
M_ = np.ma.array(M, mask=mascara)

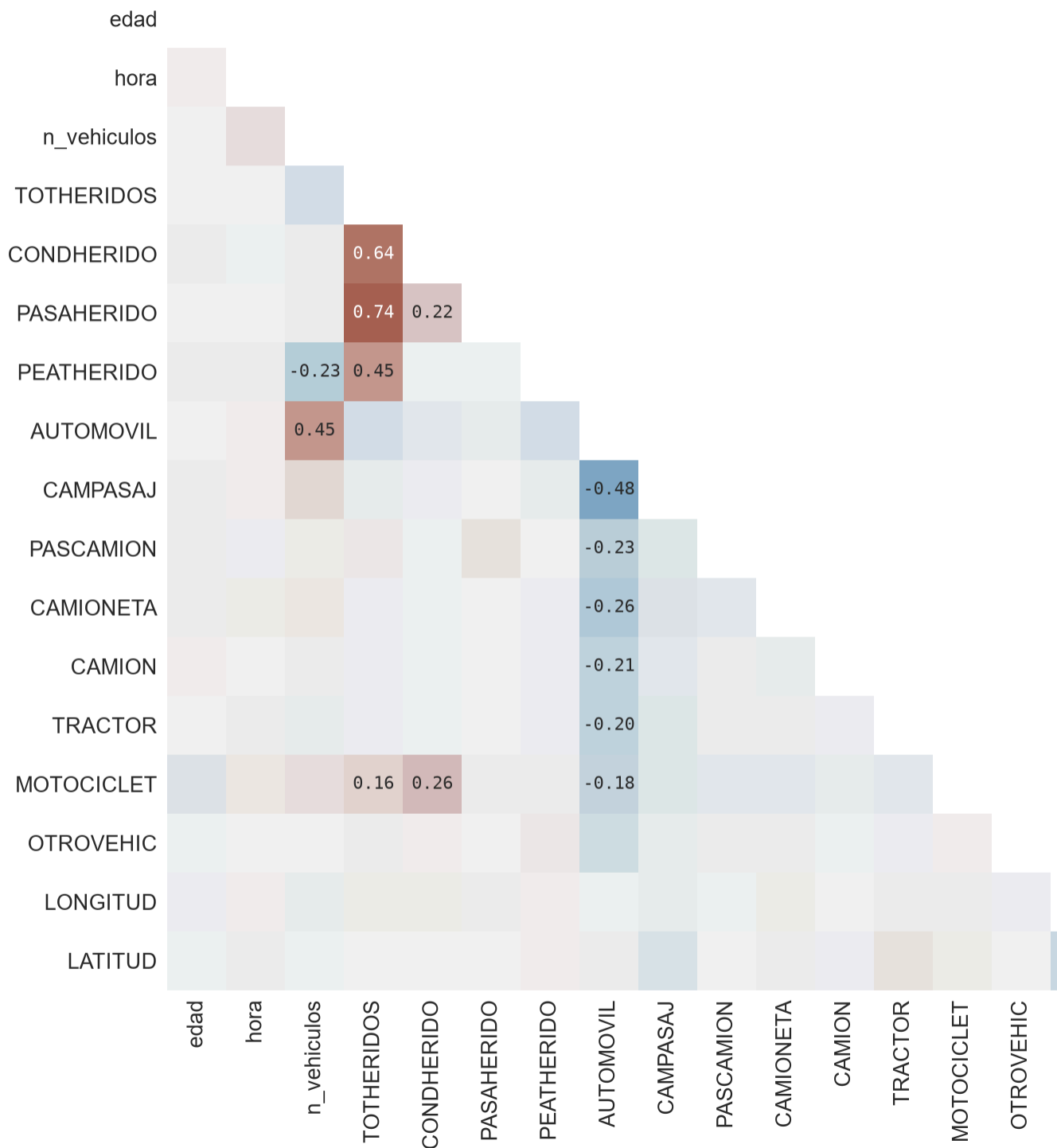
im = ax.imshow(M_, cmap=DIVERGENTE, vmin=-1, vmax=1)
ax.set_xticks(range(len(CUANT)), CUANT, rotation=90)
ax.set_yticks(range(len(CUANT)), CUANT)

# Etiqueta directa solo donde hay algo que leer, nunca en las 136 celdas.
for i in range(len(CUANT)):
    for j in range(i):
        if abs(M[i, j]) >= .15:
            ax.text(j, i, f"{M[i, j]:.2f}", ha="center", va="center",
                    family="monospace", fontsize=6.2,
                    color=BLANCO if abs(M[i, j]) > .55 else GRAFITO)

cb = fig.colorbar(im, ax=ax, shrink=.62, ticks=[-1, -.5, 0, .5, 1])
cb.set_label("r de Pearson", fontsize=8.5)
cb.outline.set_visible(False)
ax.grid(False)
ax.set_title("Correlación lineal")
ax.tick_params(length=0, labelsize=7.6)
for lado in ("top", "right", "left", "bottom"):
    ax.spines[lado].set_visible(False)
fig.text(0.005, -0.01, "Solo se anotan los pares con  $|r| \geq 0.15$ . Gris = sin asociación.",
        fontsize=8, color=GRAFITO, family="serif")
plt.show()

```

Correlación lineal



Solo se anotan los pares con $|r| \geq 0.15$. Gris = sin asociación.

```
# Pearson contra Spearman: una sola serie, así que un solo tono. Lo que se lee
# es la distancia a la diagonal, no la identidad de cada punto.
pe, sp = pares(PEARSON), pares(SPEARMAN)
comun = pe.index.intersection(sp.index)
```

```

brecha = (sp[comun] - pe[comun]).abs().sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(6.4, 6.0))
ax.axline((0, 0), slope=1, color=GRAFITO, lw=1, ls=(0, (4, 3)), zorder=2)
ax.scatter(pe[comun], sp[comun], s=30, color=AZUL, alpha=.65,
           edgecolors="none", zorder=3)

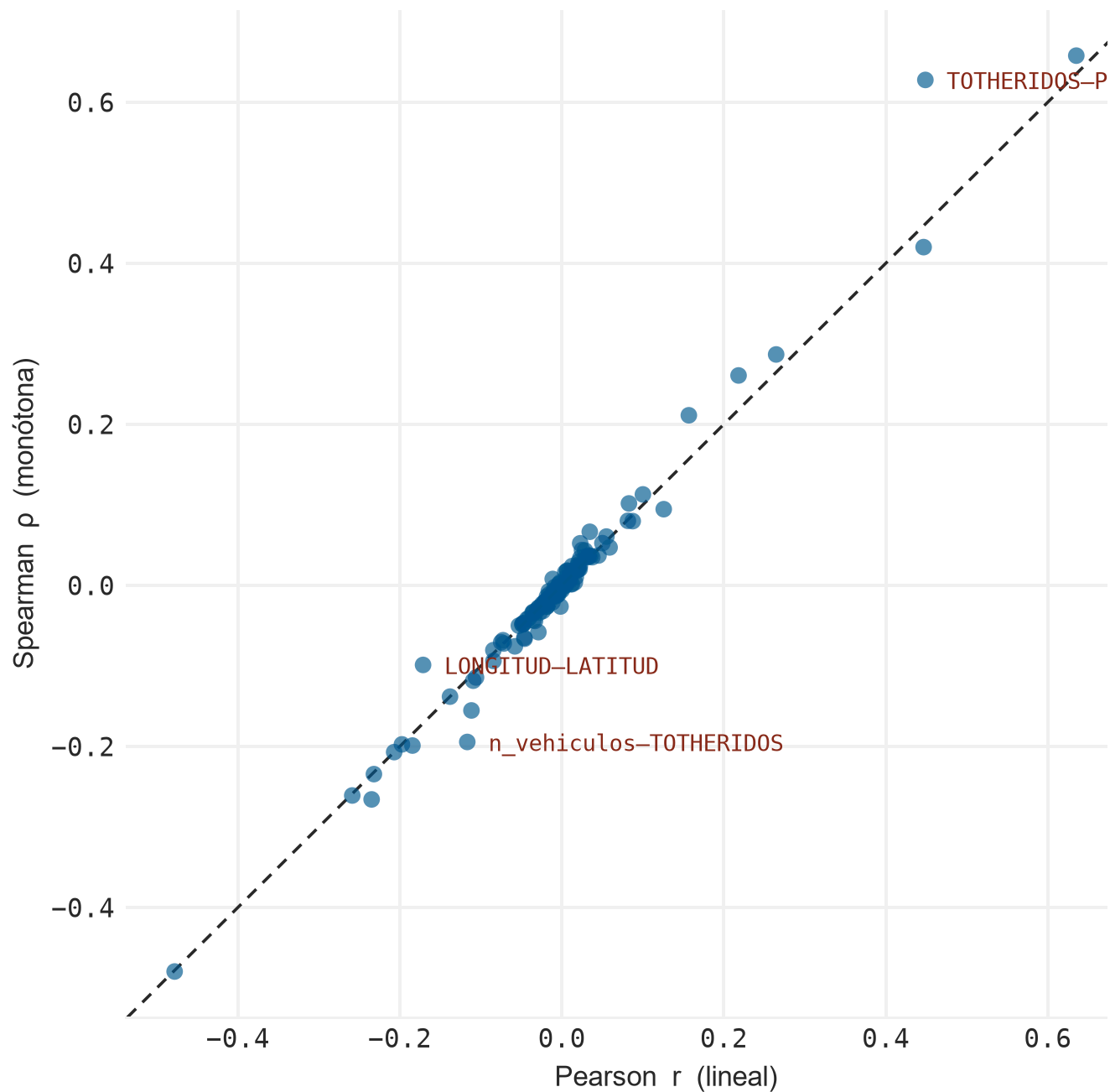
for par in brecha.head(4).index:
    ax.annotate(f"{par[0]}-{par[1]}", (pe[par], sp[par]),
               xytext=(7, -3), textcoords="offset points",
               family="monospace", fontsize=7.4, color=ROJO)

ax.set_xlabel("Pearson r (lineal)"); ax.set_ylabel("Spearman ρ (monótona)")
ax.set_title("Dónde discrepan los dos coeficientes")
pulir(ax)
fig.text(0.005, -0.02, "Cada punto es uno de los 136 pares. Sobre la diagonal, "
        "ambos coinciden; lejos, la relación es monótona pero no lineal.",
        fontsize=8, color=GRAFITO, family="serif")
plt.show()

print("Mayores discrepancias |p - r|:")
print(pd.DataFrame({"Pearson": pe[brecha.head(6).index].round(3),
                    "Spearman": sp[brecha.head(6).index].round(3),
                    "|dif|": brecha.head(6).round(3)}).to_string())

```

Dónde discrepan los dos coeficientes



Cada punto es uno de los 136 pares. Sobre la diagonal, ambos coinciden; lejos, la relación es monótona per



Mayores discrepancias $|\rho - r|$:

		Pearson	Spearman	dif
TOTHERIDOS	PASAHERIDO	0.735	0.461	0.274
	PEATHERIDO	0.449	0.628	0.179
n_vehiculos	TOTHERIDOS	-0.117	-0.195	0.078
LONGITUD	LATITUD	-0.171	-0.099	0.073
TOTHERIDOS	MOTOCICLET	0.157	0.211	0.054
	AUTOMOVIL	-0.112	-0.155	0.044

La matriz es casi toda gris: **la estructura lineal entre estas variables es débil**, y eso ya es un resultado. Los bloques que aparecen son mecánicos, no descubrimientos:

- **TOTHERIDOS** con **CONDHERIDO**, **PASAHERIDO** y **PEATHERIDO** — el total es la suma de sus componentes, así que la correlación está garantizada por construcción.
- **n_vehiculos** con **AUTOMOVIL** — mismo caso: **n_vehiculos** es la suma de los trece conteos y el automóvil aparece en el 83 % de los accidentes.
- **AUTOMOVIL** con **MOTOCICLET** o **CAMPASAJ** en negativo — competencia por el mismo lugar: si el vehículo involucrado fue una moto, no fue un automóvil.

Las discrepancias entre Pearson y Spearman se concentran en los pares que incluyen conteos con muchos ceros, donde la relación es monótona pero escalonada y el coeficiente lineal la subestima. Con este perfil de empates, **Kendall es el coeficiente defendible** y Pearson el que hay que leer con más cautela.

7. Cualitativas: one-hot y V de Cramér

Las variables realmente informativas de esta base son categóricas. Para meterlas al análisis se codifican en indicadores binarios (*one-hot*), con dos decisiones que no son de trámite.

Los ausentes son su propia categoría. **sexo_conductor** y **aliento_alcoholico** tienen 36,973 y 55,639 nulos, y el notebook de calidad mostró que ese faltante es **MNAR**: depende de la gravedad. Tratarlo como dato perdido borraría información; se codifica como nivel **NA**.

Hay que dejar fuera una categoría por variable. Si se codifican todas, los indicadores de cada variable suman exactamente 1 y la matriz queda singular — sin inversa, y sin inversa no hay factorial ni correlación parcial.

```
CUAL = ["gravedad", "ubicacion", "sexo_conductor", "dia_semana",
        "aliento_alcoholico", "TIPACCID", "CAUSAACCI"]

cat = pd.DataFrame({c: d[c].astype("string") for c in CUAL})
cat = cat.fillna("NA") # el ausente es un nivel, no un hueco

for c in CUAL:
    print(f" {c:<20} {cat[c].nunique():>2} niveles")

print()
for drop in (False, True):
    D = pd.get_dummies(cat, drop_first=drop).astype("float64")
    rango = np.linalg.matrix_rank(D.to_numpy())
    print(f" drop_first={str(drop):<5} -> {D.shape[1]:>2} columnas, rango {rango}"
          f" {'SINGULAR' if rango < D.shape[1] else 'rango completo'}")

ONEHOT = pd.get_dummies(cat, drop_first=True).astype("float64")
```

gravedad	3 niveles
ubicacion	5 niveles
sexo_conductor	3 niveles
dia_semana	7 niveles
aliento_alcoholico	3 niveles
TIPACCID	12 niveles
CAUSAACCI	5 niveles

drop_first=False -> 38 columnas, rango 32 SINGULAR
 drop_first=True -> 31 columnas, rango 31 rango completo

```
def cramers_v(x, y):
    """V de Cramér: asociación entre dos categóricas, en [0, 1]."""
    tabla = pd.crosstab(x, y)
    chi2 = stats.chi2_contingency(tabla).statistic
    n = tabla.to_numpy().sum()
    r, k = tabla.shape
    return np.sqrt((chi2 / n) / max(min(r - 1, k - 1), 1))

V = pd.DataFrame(
    [[cramers_v(cat[a], cat[b]) for b in CUAL] for a in CUAL],
    index=CUAL, columns=CUAL,
)

pares_v = V.where(np.triu(np.ones(V.shape), 1).astype(bool)).stack()
print("Pares con mayor asociación:")
print(pares_v.sort_values(ascending=False).head(6).round(3).to_string())

fig, ax = plt.subplots(figsize=(6.6, 5.4))
M = V.to_numpy().copy()
M_ = np.ma.array(M, mask=np.triu(np.ones_like(M, dtype=bool)))

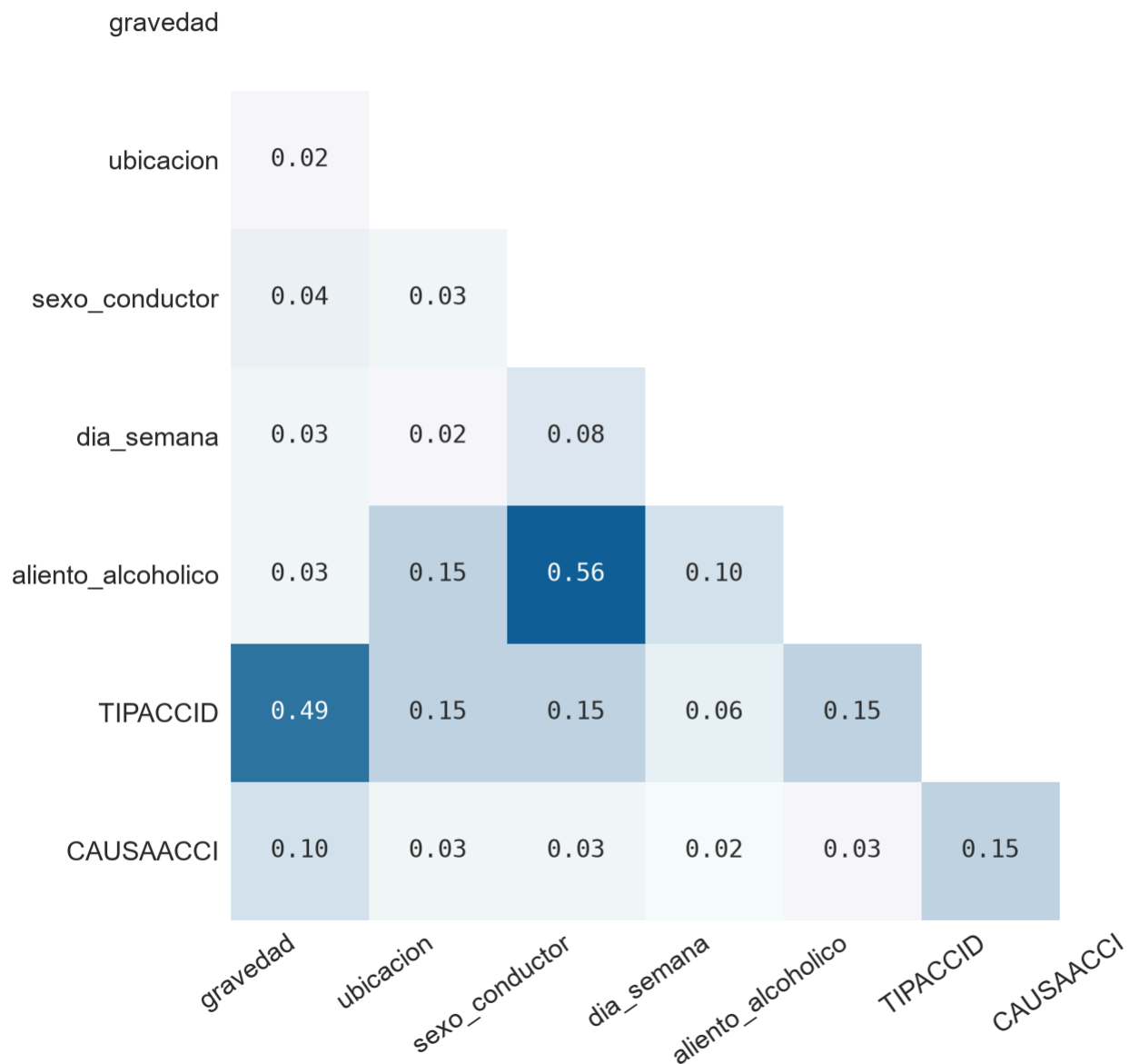
# V no tiene signo: es magnitud pura -> escala secuencial de un solo tono.
im = ax.imshow(M_, cmap=SECUENCIAL, vmin=0, vmax=.6)
ax.set_xticks(range(len(CUAL)), CUAL, rotation=35, ha="right")
ax.set_yticks(range(len(CUAL)), CUAL)
for i in range(len(CUAL)):
    for j in range(i):
        ax.text(j, i, f"{M[i, j]:.2f}", ha="center", va="center",
                family="monospace", fontsize=7.6,
                color=BLANCO if M[i, j] > .34 else GRAFITO)
cb = fig.colorbar(im, ax=ax, shrink=.7); cb.set_label("V de Cramér", fontsize=8.5)
cb.outline.set_visible(False)
ax.grid(False); ax.tick_params(length=0, labelsize=8)
for lado in ("top", "right", "left", "bottom"):
    ax.spines[lado].set_visible(False)
ax.set_title("Asociación entre cualitativas")
fig.text(0.005, -0.04, "V de Cramér, no  $\chi^2$ : con n = 379,294 todos los p valen 0 y "
        "lo único legible es el tamaño del efecto.", fontsize=8, color=GRAFITO,
```

```
family="serif")
plt.show()
```

Pares con mayor asociación:

sexo_conductor	aliento_alcoholico	0.561
gravedad	TIPACCID	0.490
sexo_conductor	TIPACCID	0.153
ubicacion	aliento_alcoholico	0.153
aliento_alcoholico	TIPACCID	0.153
ubicacion	TIPACCID	0.151

Asociación entre cualitativas



V de Cramér, no χ^2 : con $n = 379,294$ todos los p valen 0 y lo único legible es el tamaño del efecto.

```
# La asociación más alta merece una comprobación antes de interpretarla.
t = pd.crosstab(d.se_fugo, d.aliento_alcoholico.isna(), normalize="index").mul(100)
t.columns = ["aliento con dato (%)", "aliento NA (%)"]
t.index = ["no se fugó", "se fugó"]
print(t.round(1).to_string())

ambos = int((d.se_fugo & d.aliento_alcoholico.isna()).sum())
print()
print(f"fugas con aliento NA: {ambos:,} de {int(d.se_fugo.sum()):,} "
      f"({ambos / d.se_fugo.sum() * 100:.1f} %)")
print(f"son el {ambos / d.aliento_alcoholico.isna().sum() * 100:.1f} % de todos "
      f"los NA de aliento")
```

	aliento con dato (%)	aliento NA (%)
no se fugó	94.5	5.5
se fugó	0.0	100.0

fugas con aliento NA: 36,973 de 36,973 (100.0 %)
 son el 66.5 % de todos los NA de aliento

Aquí sí hay estructura, y bastante más fuerte que entre las cuantitativas. Pero la asociación más alta **no es un hallazgo, es un artefacto**, y conviene desmontarla antes de que alguien la cite.

`sexo_conductor` ↔ `aliento_alcoholico` da $V = 0.561$, la más alta del conjunto. La comprobación de arriba explica por qué: **el 100 % de las 36,973 fugas tiene NA en aliento**, y esos casos son el 66.5 % de todos los ausentes de esa variable. Si el conductor huyó, no hay a quién hacerle la prueba. Lo que mide esa celda es que dos variables comparten el mismo mecanismo de faltante, no que el sexo del conductor tenga que ver con el alcohol.

Descontada esa, **la asociación real más fuerte es** `TIPACCID` ↔ `gravedad` ($V = 0.490$): el tipo de accidente predice la severidad mucho mejor que cualquier variable continua de la §6. Atropellar a un peatón y rozar un poste no son el mismo evento con distinta suerte.

El resto cae por debajo de 0.16. Notablemente, `gravedad` ↔ `día_semana` es 0.032 y `gravedad` ↔ `ubicacion` 0.022: **cuándo y dónde ocurre el accidente no dice casi nada sobre qué tan grave será**. Lo que lo determina es el tipo de choque.

8. Análisis factorial

Antes de factorizar hay que comprobar que la matriz de correlación **tiene algo que factorizar**:

- **Bartlett**: contrasta si la matriz es la identidad. Si no se rechaza, las variables son independientes y no hay factores comunes que extraer.
- **KMO**: cuánta de la correlación observada es común y no de pares aislados. Bajo 0.5 se considera inaceptable; sobre 0.8, bueno.

```

def kmo(R):
    """Kaiser-Meyer-Olkin global y por variable (MSA)."""
    Rinv = np.linalg.pinv(R)
    dd = np.sqrt(np.diag(Rinv))
    P = -Rinv / np.outer(dd, dd)          # correlaciones parciales
    np.fill_diagonal(P, 0)
    Rf = R.copy(); np.fill_diagonal(Rf, 0)
    return ((Rf ** 2).sum() / ((Rf ** 2).sum() + (P ** 2).sum()),
            (Rf ** 2).sum(0) / ((Rf ** 2).sum(0) + (P ** 2).sum(0)))

def bartlett(R, n):
    """Esfericidad de Bartlett. H0: la matriz de correlación es la identidad."""
    p = R.shape[0]
    _, logdet = np.linalg.slogdet(R)
    chi2 = -((n - 1) - (2 * p + 5) / 6) * logdet
    gl = p * (p - 1) / 2
    return chi2, gl, stats.chi2.sf(chi2, gl)

R = PEARSON.to_numpy()
n_ef = len(Xp.dropna())
chi2, gl, p_b = bartlett(R, n_ef)
kmo_g, kmo_v = kmo(R)

print(f"Bartlett:  $\chi^2$  = {chi2:,.0f}   gl = {gl:.0f}   p = {p_b:.3g}")
print(f"KMO global = {kmo_g:.3f}\n")
print("KMO por variable (MSA), las cinco más bajas:")
print(pd.Series(kmo_v, index=CUANT).sort_values().head(5).round(3).to_string())

```

Bartlett: χ^2 = 2,402,771 gl = 136 p = 0
KMO global = 0.060

KMO por variable (MSA), las cinco más bajas:

OTROVEHIC	0.005
CAMION	0.009
TRACTOR	0.010
PASCAMION	0.012
CAMIONETA	0.014

```

Z = (Xp - Xp.mean()) / Xp.std()          # estandarizado: la covarianza no sirve
Zc = Z.dropna()

pca = PCA().fit(Zc)
varexp = pca.explained_variance_ratio_ * 100
acum = varexp.cumsum()
k_kaiser = int((pca.explained_variance_ > 1).sum())    # criterio de Kaiser

fig, (a1, a2) = plt.subplots(1, 2, figsize=(9.6, 3.6))

```

```

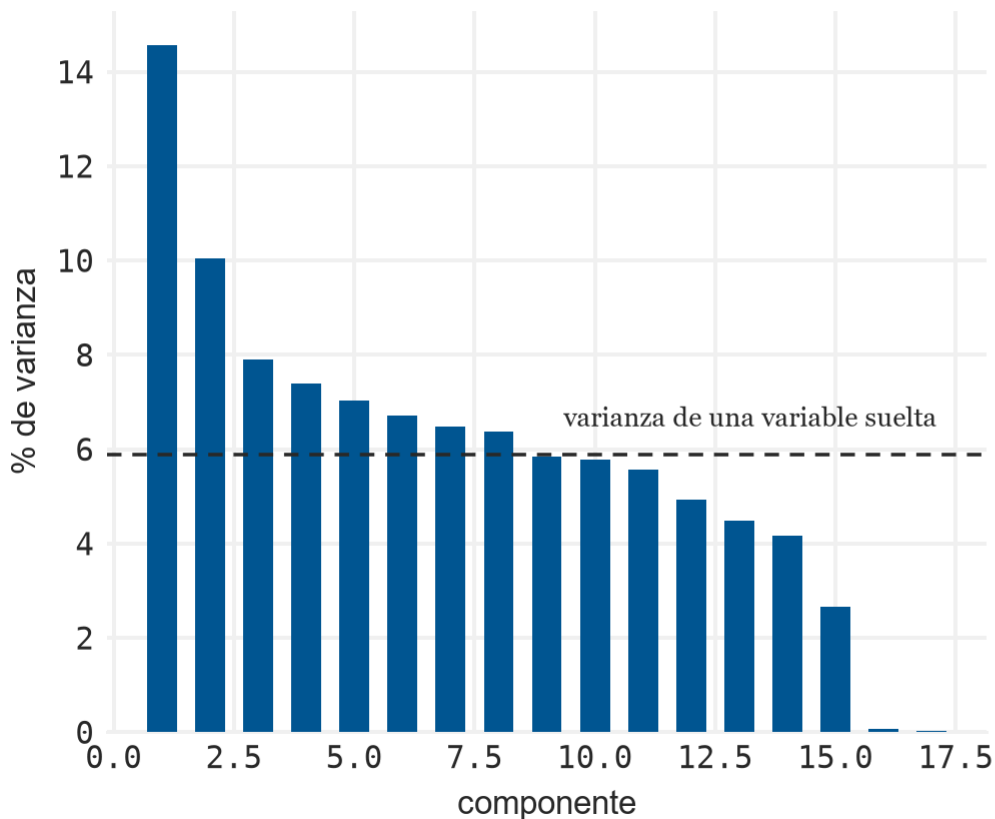
x = np.arange(1, len(varexp) + 1)
a1.bar(x, varexp, color=AZUL, width=.62, zorder=3)
a1.axhline(100 / len(CUANT), color=GRAFITO, lw=1, ls=(0, (4, 3)), zorder=4)
a1.text(len(x) * .55, 100 / len(CUANT) + .6, "varianza de una variable suelta",
        family="serif", fontsize=7.6, color=GRAFITO)
a1.set_xlabel("componente"); a1.set_ylabel("% de varianza")
a1.set_title("Sedimentación")
pulir(a1)

a2.plot(x, acum, color=AZUL, marker="o", ms=4, zorder=3)
a2.axhline(80, color=ROJO, lw=1, ls=(0, (4, 3)), zorder=2)
a2.text(1, 82, "80 %", family="monospace", fontsize=8, color=ROJO)
a2.set_xlabel("componentes"); a2.set_ylabel("% acumulado")
a2.set_title("Varianza acumulada")
pulir(a2)
plt.show()

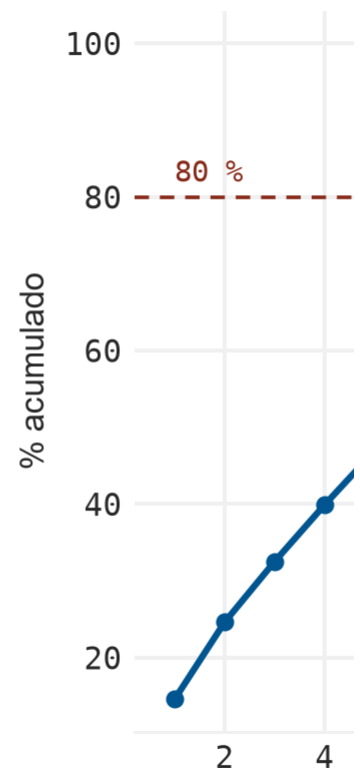
print(f"Componentes con eigenvalor > 1 (Kaiser): {k_kaiser}")
print(f"Componentes para el 80 % de la varianza: {int((acum < 80).sum()) + 1}")
print(f"El primero explica {varexp[0]:.1f} %; los tres primeros, {acum[2]:.1f} %.")

```

Sedimentación



Varianza acumulada



Componentes con eigenvalor > 1 (Kaiser): 8
 Componentes para el 80 % de la varianza: 11
 El primero explica 14.6 %; los tres primeros, 32.5 %.

```

def varimax(L, gamma=1.0, iteraciones=100, tol=1e-6):
    """Rotación varimax: busca estructura simple en las cargas."""
    L = np.asarray(L, dtype=float)
    p, k = L.shape
    Rot = np.eye(k)
    suma = 0.0
    for _ in range(iteraciones):
        previa = suma
        Lam = L @ Rot
        u, s, vt = np.linalg.svd(
            L.T @ (Lam ** 3 - (gamma / p) * Lam @ np.diag(np.diag(Lam.T @ Lam))))
        Rot = u @ vt
        suma = s.sum()
        if previa != 0 and suma / previa < 1 + tol:
            break
    return L @ Rot

K, MAX_ITER = 4, 1000
with warnings.catch_warnings(record=True) as avisos:
    warnings.simplefilter("always")
    fa = FactorAnalysis(n_components=K, max_iter=MAX_ITER, random_state=0).fit(Zc)
convergio = not any("did not converge" in str(a.message) for a in avisos)
print(f"iteraciones usadas: {len(fa.loglike_)} de {MAX_ITER} | "
      f"convergió: {convergio}")

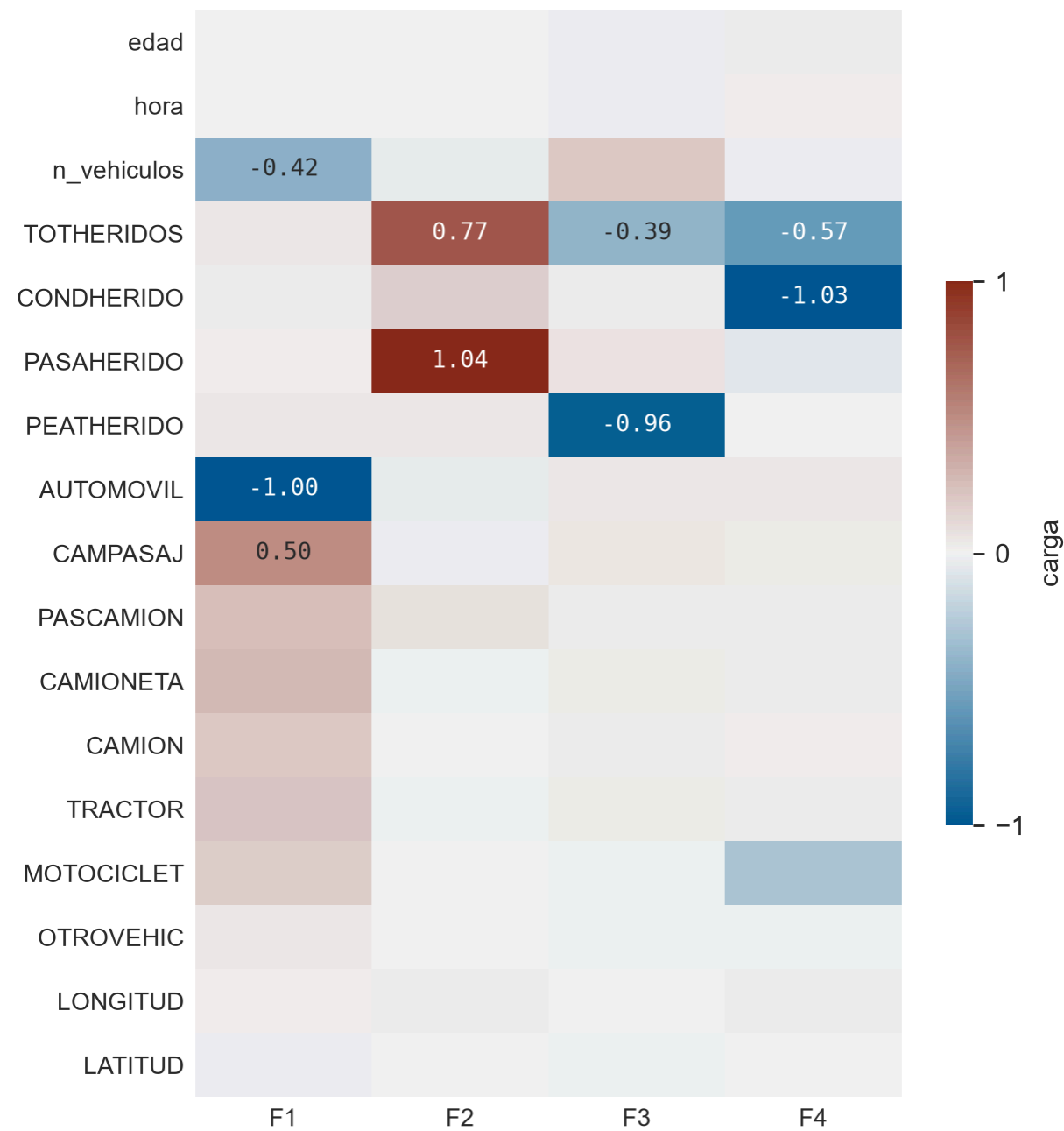
cargas = pd.DataFrame(varimax(fa.components_.T),
                      index=CUANT, columns=[f"F{i+1}" for i in range(K)])

fig, ax = plt.subplots(figsize=(5.0, 6.2))
im = ax.imshow(cargas.to_numpy(), cmap=DIVERGENTE, vmin=-1, vmax=1, aspect="auto")
ax.set_xticks(range(K), cargas.columns)
ax.set_yticks(range(len(CUANT)), CUANT)
for i in range(len(CUANT)):
    for j in range(K):
        v = cargas.iat[i, j]
        if abs(v) >= .30:
            # solo las cargas que definen el factor
            ax.text(j, i, f"{v:.2f}", ha="center", va="center", family="monospace",
                    fontsize=7.4, color=BLANCO if abs(v) > .55 else GRAFITO)
cb = fig.colorbar(im, ax=ax, shrink=.5, ticks=[-1, 0, 1])
cb.set_label("carga", fontsize=8.5); cb.outline.set_visible(False)
ax.grid(False); ax.tick_params(length=0, labelsz=7.8)
for lado in ("top", "right", "left", "bottom"):
    ax.spines[lado].set_visible(False)
ax.set_title("Cargas tras rotación varimax")
fig.text(0.005, -0.02, "Solo se anotan las cargas con |valor| ≥ 0.30.",
        fontsize=8, color=GRAFITO, family="serif")
plt.show()

cargas.round(3)

```

Cargas tras rotación varimax



Solo se anotan las cargas con |valor| ≥ 0.30.

	F1	F2	F3	F4
edad	0.005	0.001	-0.018	0.028
hora	-0.007	-0.001	-0.022	0.009
n_vehiculos	-0.421	-0.039	0.204	-0.022

	F1	F2	F3	F4
TOTHERIDOS	0.044	0.770	-0.388	-0.569
CONDHERIDO	0.019	0.170	0.025	-1.033
PASAHERIDO	0.014	1.041	0.067	-0.065
PEATHERIDO	0.050	0.054	-0.956	-0.007
AUTOMOVIL	-0.996	-0.034	0.048	0.039
CAMPASAJ	0.500	-0.018	0.061	0.034
PASCAMION	0.250	0.079	0.022	0.030
CAMIONETA	0.272	-0.011	0.033	0.018
CAMION	0.196	-0.007	0.024	0.013
TRACTOR	0.216	-0.011	0.033	0.018
MOTOCICLET	0.179	0.002	-0.009	-0.279
OTROVEHIC	0.054	0.002	-0.012	-0.013
LONGITUD	0.012	0.020	-0.007	-0.030
LATITUD	-0.023	0.005	-0.009	0.005

Bartlett rechaza ($\chi^2 = 2,402,771$) y el **KMO global es 0.060**. El umbral de inaceptabilidad está en 0.50, así que no es que quede corto: está a una décima parte del mínimo. Varias variables tienen un MSA individual por debajo de 0.015.

Las dos cosas juntas son coherentes: existe *alguna* correlación —Bartlett la detecta— pero está concentrada en pares aislados en vez de repartida en factores comunes, que es justo lo que el KMO mide. Con este n , Bartlett iba a rechazar pasara lo que pasara; **el que manda es el KMO**, y dice que estas variables no son material para un factorial.

El gráfico de sedimentación lo confirma: no hay codo. La varianza se reparte casi uniformemente entre los componentes, que es la firma de variables poco relacionadas. Hacen falta 11 componentes para el 80 % de la varianza de 17 variables — una reducción de dimensión que no reduce nada.

El ajuste ni siquiera converge en 1,000 iteraciones, y eso no es un problema de configuración: es el síntoma que se espera cuando no hay estructura común que estimar. Con un KMO de 0.060, el algoritmo busca factores que no existen. Se reporta el resultado no convergido a propósito, porque esconderlo subiendo el número de iteraciones daría la falsa impresión de un modelo válido.

Aun así, los factores que salen son legibles, y reproducen los bloques mecánicos de la §6 en vez de dimensiones latentes: F1 opone AUTOMOVIL (−1.00) a los demás tipos de vehículo, F2 recoge PASAHERIDO y TOTHERIDOS, F3 es PEATHERIDO casi en solitario y F4 es CONDHERIDO. Son los totales y sus componentes otra vez.

Digno de nota: edad, hora, LONGITUD y LATITUD cargan por debajo de 0.03 en los cuatro factores. **Las únicas variables que se parecían a continuas no comparten varianza con nada.**

Lectura honesta: el análisis factorial no aplica a esta base. No porque esté mal ejecutado, sino porque las variables cuantitativas del ATUS son conteos casi independientes de sucesos raros. El resultado negativo es el resultado.

9. Serie de tiempo: descomposición STL

STL (*Seasonal-Trend decomposition using Loess*) separa una serie en tendencia, estacionalidad y residuo. A diferencia de la descomposición clásica, deja que la componente estacional **cambie de forma a lo largo del tiempo**, que es lo que hace falta aquí: el patrón semanal de 2020 no es el de 2019.

Antes hay que construir la serie, y ahí aparece un problema.

```
diaria = d.groupby("fecha").size()
rango = pd.date_range(d.fecha.min(), d.fecha.max(), freq="D")
faltan = rango.difference(diaria.index)

print(f"días en el rango: {len(rango):,} | días con registros: {len(diaria):,}")
print(f"ausentes: {[str(f.date()) for f in faltan]}\n")

vecindad = diaria.reindex(pd.date_range("2024-02-24", "2024-03-03"))
print("alrededor de la fecha ausente:")
print(vecindad.to_string())
print(f"\nmedia diaria de la serie: {diaria.mean():.1f}")
print(f"máximo de los 2,192 días: {diaria.max()} el {diaria.idxmax():%Y-%m-%d}")
print(f"el 2020-02-29 (el otro año bisiesto) sí tiene datos: "
      f"{int(diaria.get(pd.Timestamp('2020-02-29'), 0))} accidentes")
```

```
días en el rango: 2,192 | días con registros: 2,191
ausentes: ['2024-02-29']
```

alrededor de la fecha ausente:

2024-02-24	198.0
2024-02-25	140.0
2024-02-26	216.0
2024-02-27	242.0
2024-02-28	439.0
2024-02-29	NaN
2024-03-01	237.0
2024-03-02	234.0
2024-03-03	170.0

Freq: D

media diaria de la serie: 173.1

máximo de los 2,192 días: 439 el 2024-02-28

el 2020-02-29 (el otro año bisiesto) sí tiene datos: 204 accidentes

El 29 de febrero de 2024 no existe en los datos, y el 28 tiene 439 accidentes: el máximo de los 2,192 días de la serie, 2.54 veces la media. En 2020, el otro año bisiesto del periodo, el 29 de febrero sí tiene sus 204 registros normales.

La lectura es que **los registros del día bisiesto de 2024 se cargaron al 28**. No estaba documentado en el diccionario ni en el notebook de calidad.

Para la descomposición se interpola el día ausente y se deja el pico del 28 tal cual. No se reparte a mano: se usa STL con estimación robusta, que reduce el peso de los valores atípicos sin borrarlos, y después se comprueba si el residuo detecta solo el 28.

```
serie = diaria.reindex(rango).interpolate("linear")
serie.index.freq = "D"

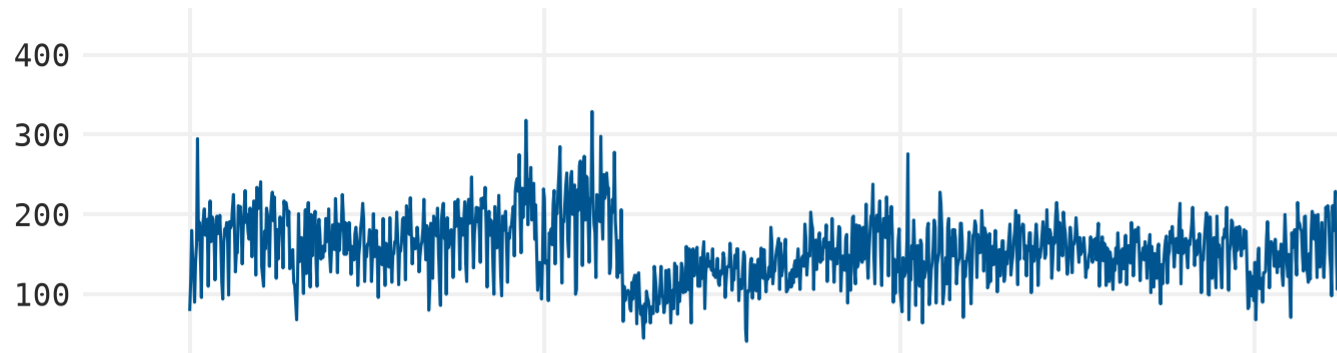
stl_d = STL(serie, period=7, robust=True).fit()

fig, ejes = plt.subplots(4, 1, figsize=(9.6, 7.4), sharex=True)
paneles = [("Serie observada", serie), ("Tendencia", stl_d.trend),
           ("Estacionalidad semanal", stl_d.seasonal), ("Residuo", stl_d.resid)]
for ax, (titulo, s) in zip(ejes, paneles):
    ax.plot(s.index, s.values, color=AZUL, lw=.85)
    ax.set_title(titulo, fontsize=9.5, color=GRAFITO, pad=6)
    pulir(ax)
ejes[3].axhline(0, color=GRAFITO, lw=.8)
ejes[3].set_xlabel("fecha")
fig.suptitle("STL diaria – periodo 7 días", x=0.005, ha="left", color=ROJO,
            fontweight="bold", fontsize=11.5)
fig.tight_layout()
plt.show()

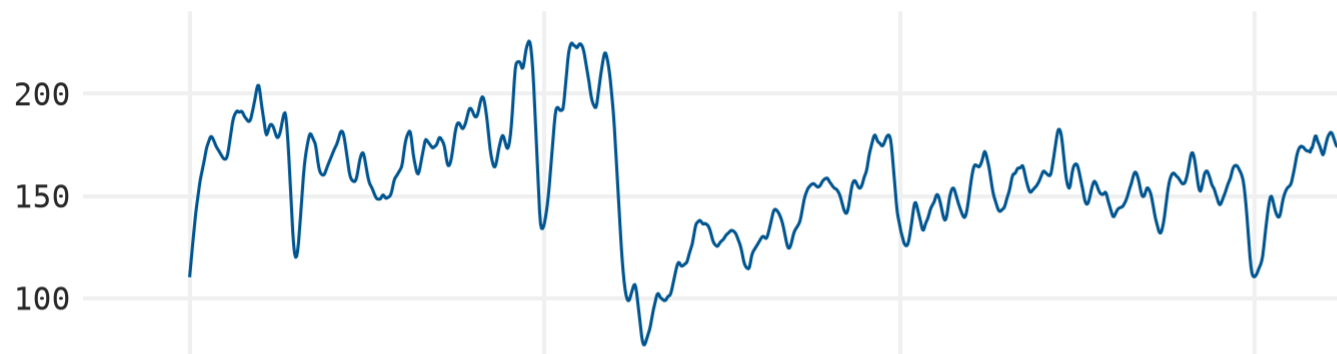
fuerza_est = max(0, 1 - stl_d.resid.var() / (stl_d.seasonal + stl_d.resid).var())
fuerza_ten = max(0, 1 - stl_d.resid.var() / (stl_d.trend + stl_d.resid).var())
print(f"fuerza de la estacionalidad: {fuerza_est:.3f}")
print(f"fuerza de la tendencia: {fuerza_ten:.3f}")
```

STL diaria — periodo 7 días

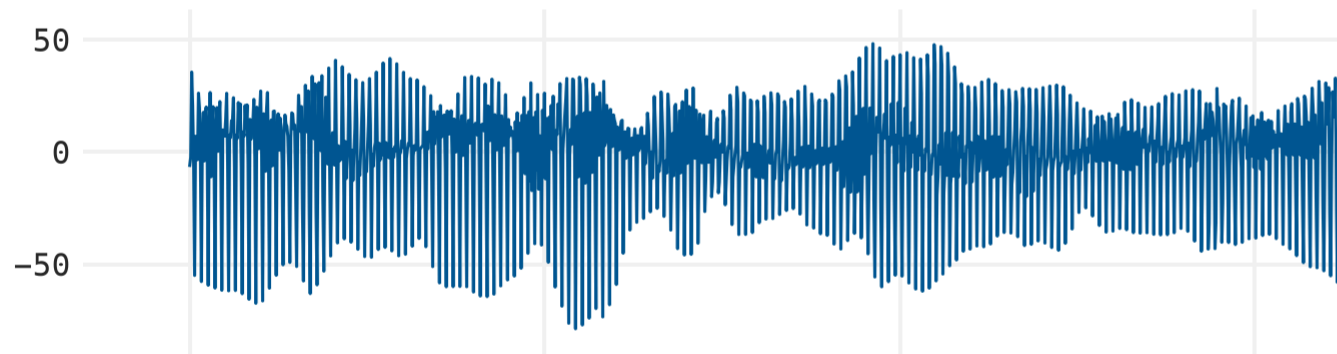
Serie observada



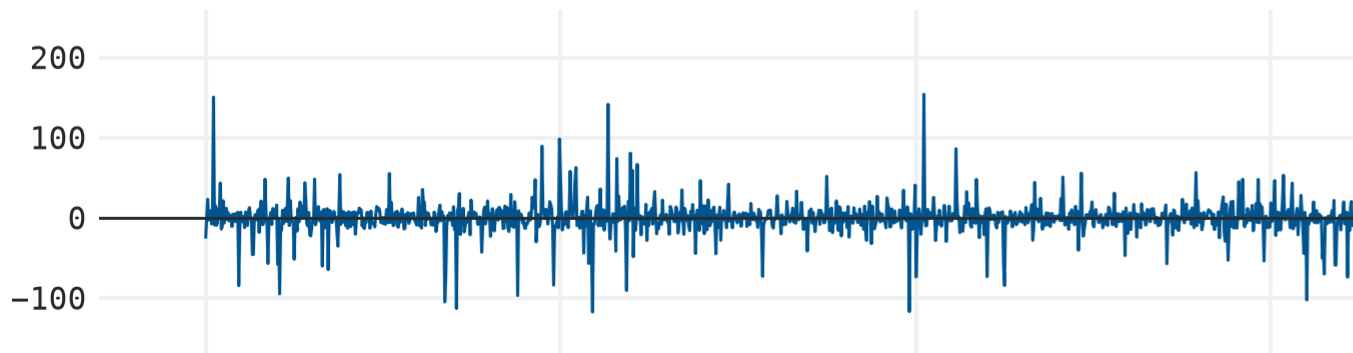
Tendencia



Estacionalidad semanal



Residuo



2019

2020

2021

2022

fecha

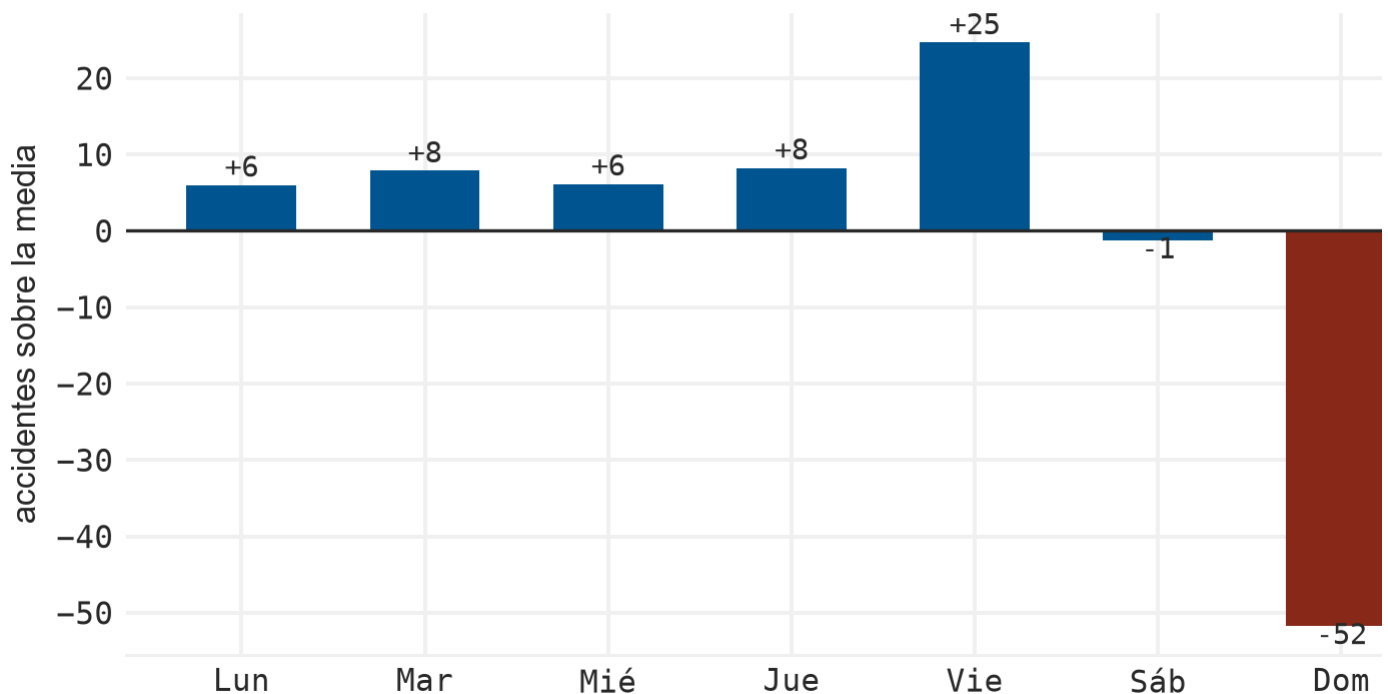
fuerza de la estacionalidad: 0.521

fuerza de la tendencia: 0.641

```
# El perfil semanal, tomado de la componente estacional.
perfil = (stl_d.seasonal.groupby(stl_d.seasonal.index.dayofweek).mean())
perfil.index = ["Lun", "Mar", "Mié", "Jue", "Vie", "Sáb", "Dom"]

fig, ax = plt.subplots(figsize=(6.6, 3.2))
# Se destaca la desviación de mayor magnitud, que es el déficit del domingo:
# más del doble que el exceso del viernes.
extremo = perfil.abs().max()
colores = [ROJO if abs(v) == extremo else AZUL for v in perfil.values]
ax.bar(perfil.index, perfil.values, color=colores, width=.6, zorder=3)
ax.axhline(0, color=GRAFITO, lw=.9, zorder=4)
for i, v in enumerate(perfil.values):
    ax.text(i, v + (1.1 if v >= 0 else -2.4), f"{v:+.0f}", ha="center",
            family="monospace", fontsize=8, color=GRAFITO)
ax.set_ylabel("accidentes sobre la media")
ax.set_title("El domingo pesa más que el viernes")
pulir(ax)
plt.show()
```

El domingo pesa más que el viernes



```
# ¿El residuo detecta el artefacto del día bisiesto?
z = (stl_d.resid - stl_d.resid.mean()) / stl_d.resid.std()
```

```
print("Diez días con el residuo más extremo:")
print(pd.DataFrame({"accidentes": serie.reindex(z.abs().nlargest(10).index).astype(int),
                    "residuo z": z.reindex(z.abs().nlargest(10).index).round(2)}).to_string())
```

Diez días con el residuo más extremo:

	accidentes	residuo z
2024-02-28	439	10.55
2024-06-20	44	-6.84
2021-01-09	276	6.80
2019-01-09	295	6.64
2020-02-19	329	6.25
2024-10-01	104	-5.84
2024-06-21	84	-5.63
2023-04-07	68	-5.51
2024-11-18	125	-5.45
2020-02-03	106	-5.12

```
# Mensual: la estacionalidad anual y el efecto de 2020 se leen mejor aquí.
mensual = d.groupby(pd.Grouper(key="fecha", freq="MS")).size().astype("float64")
mensual.index.freq = "MS"
stl_m = STL(mensual, period=12, robust=True).fit()

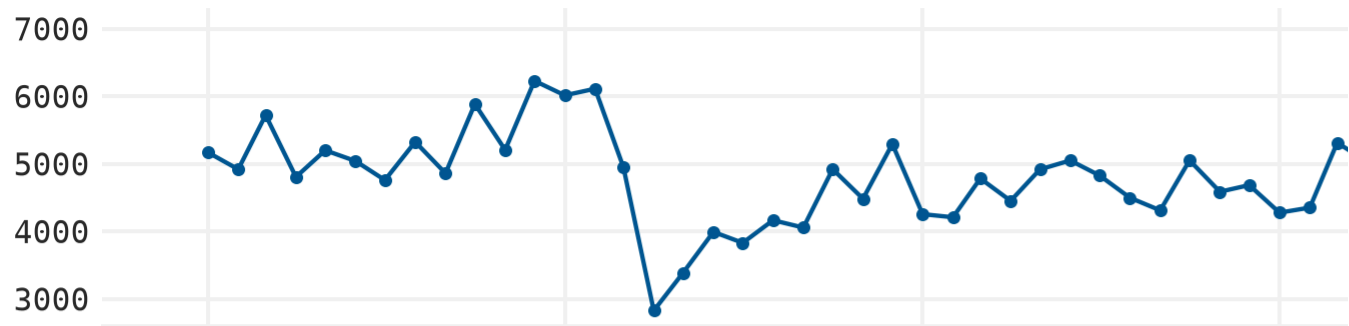
fig, ejes = plt.subplots(4, 1, figsize=(9.6, 7.0), sharex=True)
paneles = [("Serie observada", mensual), ("Tendencia", stl_m.trend),
            ("Estacionalidad anual", stl_m.seasonal), ("Residuo", stl_m.resid)]
for ax, (titulo, s) in zip(ejes, paneles):
    ax.plot(s.index, s.values, color=AZUL, marker="o", ms=2.6, lw=1.2)
    ax.set_title(titulo, fontsize=9.5, color=GRAFITO, pad=6)
    pulir(ax)

# La caída de 2020 se marca una sola vez, en el panel de la tendencia.
ejes[1].axvspan(pd.Timestamp("2020-03-01"), pd.Timestamp("2020-12-31"),
                color=GRIS, zorder=0)
ejes[1].text(pd.Timestamp("2020-03-15"), ejes[1].get_ylim()[1] * .97, "2020",
             family="monospace", fontsize=8, color=ROJO, va="top")
ejes[3].axhline(0, color=GRAFITO, lw=.8)
ejes[3].set_xlabel("mes")
fig.suptitle("STL mensual – periodo 12 meses", x=0.005, ha="left", color=ROJO,
             fontweight="bold", fontsize=11.5)
fig.tight_layout()
plt.show()

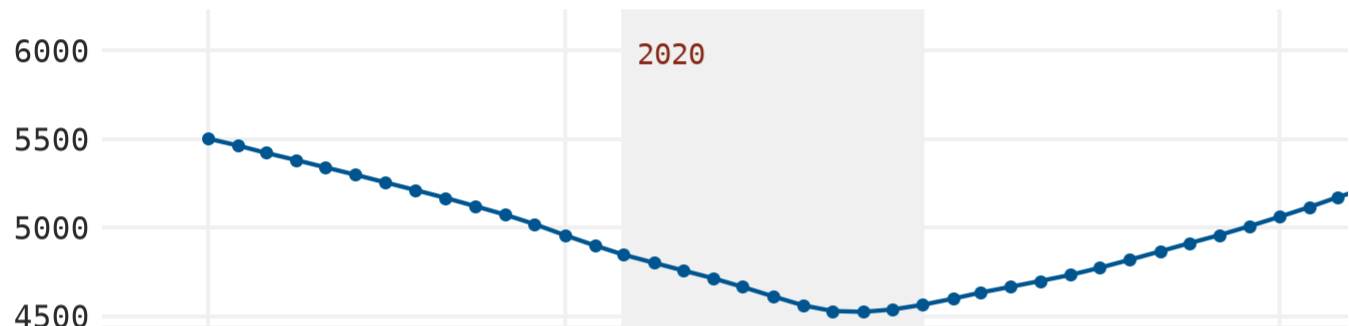
print("accidentes por año:")
print(d.groupby(d.fecha.dt.year).size().to_string())
```

STL mensual — periodo 12 meses

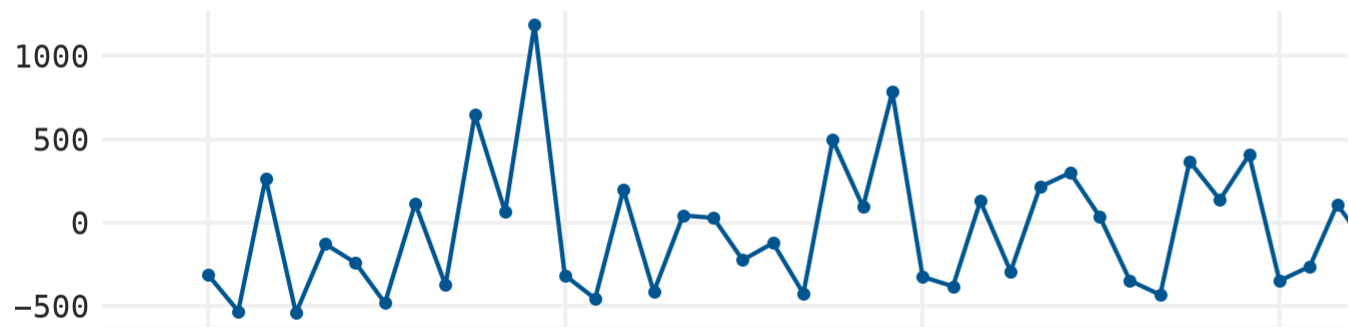
Serie observada



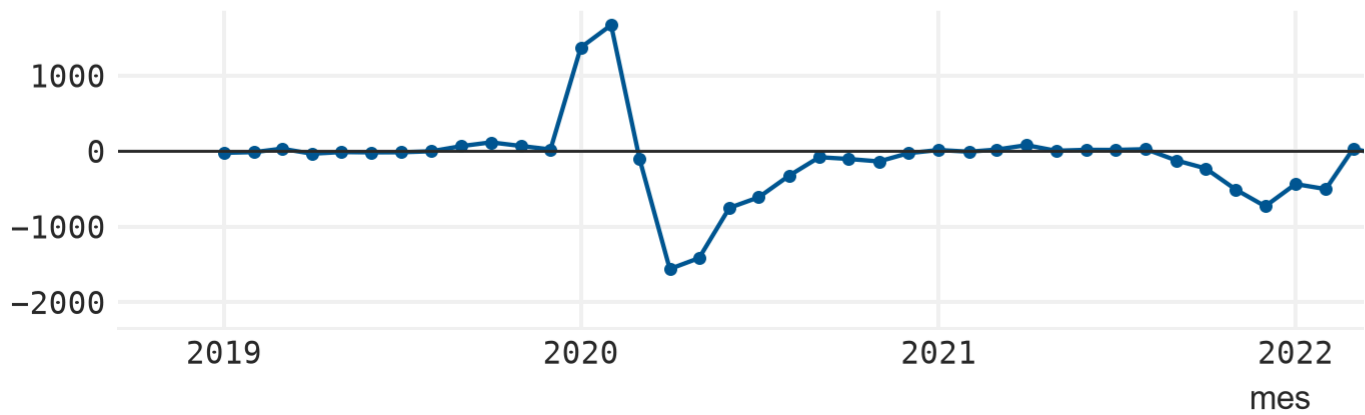
Tendencia



Estacionalidad anual



Residuo



accidentes por año:

fecha

2019 63134

2020 54041

2021 55639

2022 65257

2023 69902

2024 71321

El patrón semanal es nítido y asimétrico. El domingo pierde 51.7 accidentes respecto a la media y el viernes gana 24.6: el déficit dominical pesa más del doble que el exceso del viernes. De lunes a jueves el efecto es plano (+6 a +8) y el sábado es prácticamente neutro (−1.2). No es una onda suave sino un escalón: el domingo se comporta distinto a todos los demás días.

La fuerza de la tendencia (0.641) **supera a la de la estacionalidad** (0.521). Los seis años de recuperación tras 2020 mueven la serie más que el ciclo semanal.

El residuo detecta el artefacto sin que se le diga dónde mirar. El día con el residuo más extremo de los 2,192 es el 2024-02-28, con $z = 10.6$ — el pico donde se cargaron los registros del día bisiesto. El segundo mayor está en $z = -6.8$, muy por detrás. Los demás días atípicos quedan como pregunta abierta: no se identifican aquí porque no hay nada en la base que permita atribuirles una causa, y ponerles una etiqueta sin evidencia sería inventar.

La tendencia mensual reconstruye la pandemia. Caída en 2020, recuperación lenta durante 2021 y superación del nivel prepandemia a partir de 2022. Los 71,321 accidentes de 2024 son un 13 % más que los 63,134 de 2019 — y aquí ese crecimiento **sí es comparable**, porque el panel de la ZMM está balanceado: los mismos 18 municipios los seis años. A nivel nacional esa comparación no se puede hacer, porque la cobertura pasa de 91 a 198 municipios.

Conclusiones

Sobre las variables. De 30 candidatas cuantitativas solo 17 tienen varianza suficiente, y de esas solo **edad** se parece a una normal. El resto son conteos de sucesos raros: **OTROMUERTO** tiene curtosis 54,180. **TRANVIA** es constante cero.

Sobre la normalidad. Se rechaza en todos los niveles: univariado (Anderson-Darling en las 17) y multivariado (Mardia y Henze-Zirkler, las ocho muestras, con b_{2p} entre 1,610 y 5,143 contra 323 esperado). Las dos pruebas se validaron antes contra datos simulados normales, donde correctamente no rechazan. Queda descartada cualquier técnica que exija normalidad multivariada.

Sobre la correlación. La estructura lineal entre las cuantitativas es débil y lo poco que hay es mecánico: totales contra sus componentes, y exclusión mutua entre tipos de vehículo. Pearson y Spearman discrepan justo en los pares con muchos ceros; con este perfil de empates, **Kendall es el coeficiente defendible**.

Sobre la factorización. Bartlett rechaza pero el KMO global es **0.060**, contra un mínimo aceptable de 0.50, y no hay codo en la sedimentación: hacen falta 11 componentes para el 80 % de la varianza de 17 variables.

El factorial no aplica aquí, y ese resultado negativo es informativo: dice que la información del ATUS no está en la covarianza entre sus conteos.

Dónde sí está la información. En las cualitativas — con una trampa. La V de Cramér más alta (0.561, `sexo_conductor` ↔ `aliento_alcoholico`) es un artefacto: las dos variables comparten mecanismo de faltante, porque al conductor que huye no se le puede hacer la prueba de aliento. La asociación real más fuerte es `TIPACCID` ↔ `gravedad` (0.490), que supera a cualquier correlación entre continuas: el tipo de accidente predice la severidad mejor que todo lo demás junto. En cambio `gravedad` ↔ `dia_semana` es 0.032 — cuándo ocurre el accidente no anticipa qué tan grave será.

Hallazgo nuevo de este notebook: el 29 de febrero de 2024 no existe en la base y sus registros están cargados en el 28, que por eso es el día con más accidentes de los seis años. El residuo del STL lo señala como la anomalía más extrema de la serie. Debería documentarse en el diccionario de datos.

Advertencia metodológica

Con $n = 379,294$ ninguna prueba de hipótesis de este notebook es informativa por su p : todas rechazan. Lo que se leyó en cada caso fue el **tamaño del efecto** — sesgo, curtosis, magnitud de las correlaciones, KMO, fuerza de la estacionalidad. Reportar « $p < 0.001$ » sobre esta base no distingue un hallazgo de una trivialidad.